# NOESIS: A PROTOCOL FOR TRADING AND FULFILLING MACHINE INTELLIGENCE AS A COMMODITY

GIACINTO PAOLO (GP) SAGGESE

Abstract. Applications and autonomous agents increasingly consume large language model (LLM) inference as a raw input, in the same way that factories consume electricity. Today this input is purchased through static, per-model price lists set unilaterally by a small number of providers, with no price discovery, no verification that the delivered service matched what was paid for, and no mechanism to route a request to only as much capability as it actually requires. We propose Noesis, a protocol that treats machine intelligence as a fungible commodity and organizes its exchange around two cooperating components. NoesisMarket is a periodic call auction that matches buyers, who bid for a bundle of task volume, capability tier, latency, and reliability, against sellers, who ask a price for delivering that bundle, clearing at a single uniform price per capability tier. NoesisServer is the fulfillment and monitoring layer: an API gateway that executes the matched contracts against real model providers, meters whether the delivered capability, latency, and reliability met the contracted terms, and feeds the result back into the market as a reputation and pricing signal. As a byproduct, NoesisServer logs every request and estimates how much capability each request actually needs, enabling difficulty-aware routing and answer fusion that trade cost for accuracy in opposite directions. We formalize the contract, bid, and ask notation; state the market-clearing problem as a linear program and establish that a clearing price exists under standard monotonicity assumptions; and describe the request-level fulfillment statistics needed to attribute a shortfall to a specific seller rather than to noise. We compare Noesis against direct provider APIs, pass-through gateways, and raw-compute spot markets, and lay out the open mechanism-design questions and a phased implementation plan for a prototype.

## Contents



*Date*: 2026-08-07.
University of Maryland, College Park, MD 20742, USA.

## 1. Introduction

Noesis is a protocol for exchanging machine intelligence, understood as LLM inference capacity delivered at a stated capability, latency, and reliability level, between buyers who need it and sellers who supply it. It brings together price discovery through an open auction, a contract that bundles

quality attributes rather than a single price-per-token, and a fulfillment layer that measures and reports whether the contract was actually honored.

1.1. **Machine intelligence as a commodity.** Electricity, natural gas, and bandwidth all share a property that makes them tradeable as commodities: any unit supplied by any seller is, for practical purposes, interchangeable with any unit supplied by any other seller, once quality attributes (voltage and frequency for electricity, calorific value for gas, latency and jitter for bandwidth) are specified and metered. LLM inference exhibits a version of the same property. A request answered at a stated capability tier, under a stated latency bound, with a stated reliability, is, from the buyer's perspective, largely interchangeable across providers that can meet those terms, even though the underlying models, weights, and hardware differ completely.

Unlike electricity, however, the traded unit cannot be reduced to a single scalar such as kilowatt-hours. Raw compute, measured in FLOPs or GPU-hours, is observable but is not what a buyer wants to purchase: two providers can spend the same compute on the same request and produce answers of very different quality, and a buyer typically does not know, or care, how much compute was spent to produce a given answer. The good that a buyer actually wants to buy is better described as a bundle: a capability level (how good the answer is expected to be), a latency bound (how long the answer takes), and a reliability level (how often the first two conditions are actually met). Section 2.3 returns to this point, which drives most of the protocol's design.

1.2. **The Noesis protocol.** NOESIS is organized around two cooperating components.

- **NoesisMarket**: a periodic call auction that matches buyers and sellers into contracts. Every $T$ minutes, buyers submit bids and sellers submit asks, bucketed by capability tier; within each tier, bids and asks are matched at a single uniform clearing price, subject to latency and reliability compatibility (Section 4).
- **NoesisServer**: the fulfillment and monitoring layer. It is an LLM API gateway, modeled on multi-provider routers such as OpenRouter[1], that executes the requests matched by a NOESISMARKET contract against the underlying providers, logs every prompt/response pair, and measures whether the delivered capability, latency, and reliability met the contract terms. Violations are reported back to NOESISMARKET as a reputation and pricing signal (Section 5).

Figure 1 lays out the resulting architecture, with the five pluggable components of Section 1.3 and Table 1 shaded in teal: the demand and supply sides submit bids and asks to NOESISMARKET's matching engine, which clears them into contracts dispatched to NOESISSERVER's API gateway. The gateway proxies matched requests to the underlying providers through per-provider APIs, and its metering logic checks the responses against the contract terms, using external intelligence-measure providers (e.g., artificialanalysis.ai[2]) as a capability reference, then reports fulfillment back to NOESISMARKET's reputation-and-feedback component, which gates the matching engine and feeds its pricing-dissemination component. The gateway can also fan a request out to several providers through the answer-fusion component (Section 5.4) when reliability outweighs cost.

[1] `https://openrouter.ai`

[2] `https://artificialanalysis.ai`

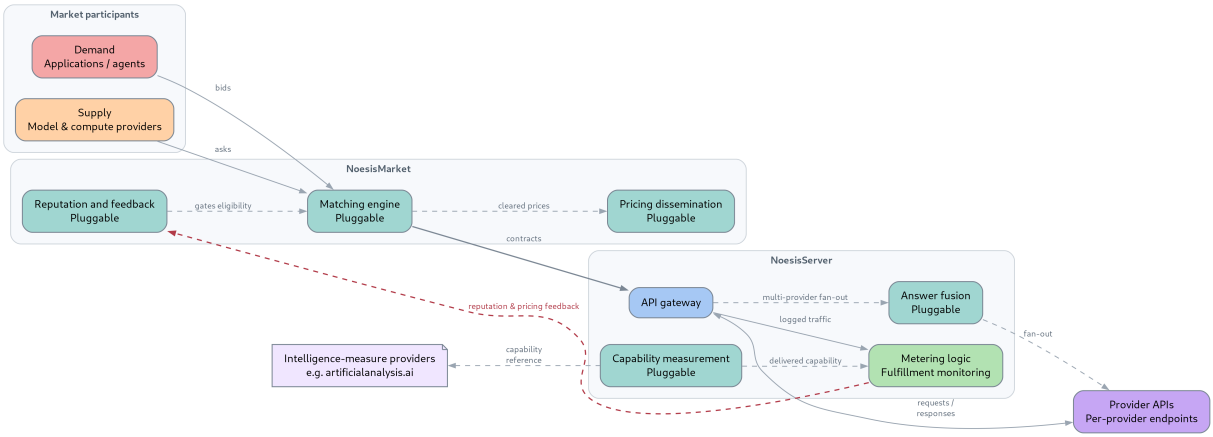


FIGURE 1. The NOESIS architecture, with its five pluggable components shaded in teal.

The two components close a loop that is missing from today's LLM API market: a contract is not merely sold, it is metered, and the metering result changes the terms under which the same seller can sell in the future. NOESISSERVER additionally reuses the logged traffic for two purposes orthogonal to fulfillment: difficulty-aware routing, which sends a request to the cheapest provider expected to satisfy it, and answer fusion, which combines several providers' answers when higher reliability is worth the extra cost.

1.3. **A modular, pluggable architecture.** The description above treats NOESISMARKET and NOESISSERVER as if each were a single, fixed implementation. In fact, each is a thin coordination layer around a small set of narrower components, each responsible for one function and exposed behind an interface that a different implementation can satisfy without requiring changes anywhere else in the pipeline:

- **Matching engine**: NOESISMARKET's only structural commitment is to match compatible bids and asks (Definition 4.2) into contracts; its responsibility stops there. The periodic uniform-price call auction of Section 4 is one instantiation of this component, and a continuous double auction or a posted-price mechanism (Section 9.1) could be substituted without touching NOESISSERVER.
- **Capability measurement**: how NOESISSERVER estimates the capability tier $\hat{c}(x)$ actually delivered by a response (Section 5.2) is itself pluggable. A verifier model, a benchmark probe, agreement with a stronger reference model (Definition 5.2), or a hosted third-party evaluation service such as artificialanalysis.ai (Section 1.2) are interchangeable implementations of the same interface: a mapping from a prompt/response pair to a tier in $\mathcal{C}$.
- **Reputation and feedback loop**: the exponential-decay update of Equation (5) is one instantiation of this component; any function that maps a seller's compliance history to a score used to gate future eligibility (Section 4.5) satisfies the same interface.
- **Answer fusion**: the aggregator $g$ of Equation (12) is already defined as a pluggable function, majority vote, a verifier model, or a learned combiner (Section 5.4); self-consistency [3] and mixture-of-agents [4] are two published instantiations of the same interface (Section 6.2).
- **Pricing dissemination**: a component absent from the description so far. NOESISMARKET clears a price $p^*(c,t)$ per tier and round (Problem 4.1), and that price is a useful signal

beyond the bidders and sellers of the round that produced it. A pricing-dissemination interface, e.g., a real-time push feed or an on-chain event log (Section 8), publishes cleared prices to subscribers, buyers timing a future bid, other market makers, monitoring dashboards, without requiring them to poll NoesisMarket or participate in a round.

Table 1 summarizes these five components, their responsibility, and the instantiation used elsewhere in this paper versus a pluggable alternative. This independence is also what lets the matching engine and the gateway-plus-logging layer be built and demonstrated as separate, independently testable pieces of a prototype, precisely because each is a separately pluggable component rather than part of one monolith.

| **Component** | **What it does** | **Example (this paper / alternative)** |
|---|---|---|
| Matching engine | Matches compatible bids and asks into contracts within a tier (Problem 4.1) | This paper: periodic uniform-price call auction (Section 4). Alternative: continuous double auction |
| Capability measurement | Estimates the capability tier $\hat{c}(x)$ actually delivered by a response (Section 5.2) | This paper: a verifier model, benchmark probe, or reference-model agreement (Definition 5.2). Alternative: a hosted evaluation provider, e.g., artificialanalysis.ai |
| Reputation and feedback loop | Aggregates a seller's fulfillment history into a score that gates future eligibility (Section 4.5) | This paper: exponential-decay update (Equation (5)). Alternative: a Bayesian trust score |
| Answer fusion | Combines several providers' answers into one when reliability outweighs cost (Section 5.4) | This paper: majority vote or a verifier model. Alternative: a learned combiner, e.g., mixture-of-agents [4] |
| Pricing dissemination | Publishes each round's cleared prices to subscribers outside the round that produced them | This paper: a real-time push feed. Alternative: an on-chain event log (Section 8) |

Table 1. The five pluggable components behind NoesisMarket and NoesisServer, their responsibility, and the instantiation used elsewhere in this paper versus a pluggable alternative.

1.4. **Contributions.** The innovative features of Noesis are:

1. **Modular, pluggable components**: NoesisMarket and NoesisServer are thin coordination layers around five independently replaceable components, matching engine, capability measurement, reputation and feedback, answer fusion, and pricing dissemination, each defined by a narrow interface rather than a fixed implementation (Section 1.3, Table 1).
2. **Price discovery through auction, not price lists**: NoesisMarket clears buyers and sellers through a periodic call auction instead of relying on prices unilaterally posted by a small number of providers.
3. **A bundled, verifiable good**: the traded unit is a (capability, latency, reliability) bundle rather than a raw compute quantity, so the market quotes and clears on "how good", not only "how much".
4. **Verified fulfillment**: NoesisServer meters every request executed under a contract and computes a measured reliability and latency, so a contract's terms are checked against evidence rather than taken on faith.
5. **Reputation and pricing feedback**: a seller whose measured performance falls short of its ask is flagged, and the resulting reputation signal feeds back into eligibility and pricing in future auction rounds, in the spirit of non-performance penalties in capacity markets.
6. **Difficulty-aware routing**: NoesisServer estimates how much capability a request actually needs and routes accordingly, spending intelligence only where it is needed.

7. **Answer fusion as a complementary mode**: for requests where reliability matters more than cost, NoesisServer can query several providers and combine their answers, trading cost for accuracy in the direction opposite to routing.
8. **Provider-agnostic liquidity pooling**: many sellers compete for the same demand behind a single interface, instead of buyers being locked into a single provider's price list.
9. **Tiered commodity pricing**: capability tiers clear independently in each auction round, analogous to peak and off-peak pricing in electricity markets.

1.5. **Comparison with alternative solutions.** Table 2 compares Noesis against three alternative ways of obtaining LLM inference: calling a provider's API directly, using a pass-through multi-provider gateway (e.g., OpenRouter-style routers), and buying raw compute on a cloud spot market (e.g., spot GPU instances). The comparison is discussed in detail in Sections 2 through 5.

| | ***Direct API*** | ***Gateway*** | ***Spot compute*** | **Noesis** |
|---|---|---|---|---|
| **Price discovery via open auction** | - | - | + | + |
| **Bundled quality guarantee** | o | o | - | + |
| **Verified fulfillment / SLA monitoring** | - | - | o | + |
| **Reputation-based accountability** | - | - | o | + |
| **Difficulty-aware cost routing** | - | o | - | + |
| **Multi-provider liquidity pooling** | - | + | o | + |
| **Reusable evaluation/training dataset** | o | o | - | + |
| **Single-API ease of integration** | + | + | - | + |

Table 2. Comparison of Noesis against alternative ways of obtaining LLM inference.

Cloud spot markets already apply auction-based price discovery, but to raw compute rather than to a capability/latency/reliability bundle, so they score poorly on the bundled-quality and difficulty-routing rows: a spot GPU market has no notion of "how good" the resulting answer is. Gateways pool liquidity across providers and are easy to integrate, but set prices unilaterally rather than clearing an auction, and typically do not meter fulfillment against a contract. Noesis is designed to combine the liquidity pooling and ease-of-integration of a gateway with the price discovery of an auction market and the verifiability of a metered service contract. Section 7 surveys the current industry landscape, startups and projects that address pieces of this same problem, in more detail.

## 2. How an intelligence market creates value

To motivate the design choices made in the rest of the paper, we step back and consider the participants, roles, and the central design difficulty of a market for machine intelligence.

2.1. **Market participants.** The demand side of the market is made of applications and autonomous agents that consume LLM inference as an input: a customer-support agent, a document-summarization pipeline, a coding assistant, or a multi-agent system delegating a sub-task. These participants differ along several dimensions:

- How much they value low latency.
- How much they value a low error rate.
- How price-sensitive they are.
- How much of their traffic is predictable in advance versus bursty.

The supply side is made of model and compute providers: foundation-model API vendors, open-weight model hosts, and resellers of spare inference capacity. Sellers differ along several dimensions:

– The capability tier of the models they can serve.
– Their typical latency and reliability under load.
– Their marginal cost of serving a request, which itself depends on hardware utilization and contractual commitments elsewhere.

As in any market, the opportunity to trade arises from these differences: a buyer with a latency-insensitive, cost-sensitive batch job and a seller with idle capacity outside of peak hours can both be made better off by a contract that a rigid, single posted price cannot express.

Figure 2 lays out both sides and the dimensions along which their participants differ: demand-side participants differ along latency sensitivity, error tolerance, price sensitivity, and traffic predictability, while supply-side participants differ along capability tier, latency/reliability under load, and marginal cost, and NoesisMarket is the meeting point where bids and asks expressing these differences are matched into contracts.

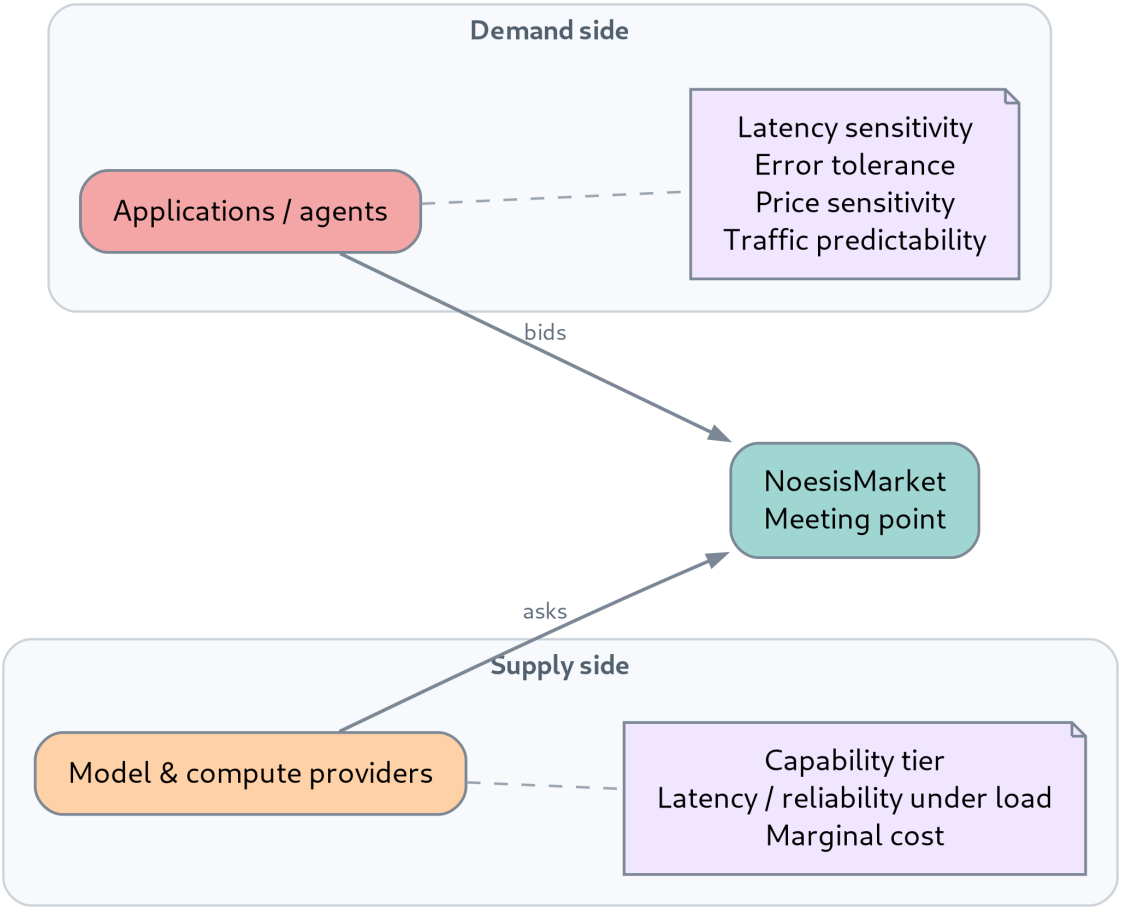


Figure 2. The two-sided Noesis market: demand and supply meeting at NoesisMarket.

2.2. **Roles of an intelligence market.** An intelligence market, like any exchange, has to perform several functions beyond simply connecting a buyer to a seller:

- Provide a common meeting ground where buyers and sellers can express demand and supply without prior bilateral negotiation.
- Establish a standardized contract schema (Section 3.3) so that a bid and an ask submitted by unrelated parties are directly comparable.
- Provide oversight by metering whether a matched contract was actually fulfilled, a role played in NOESIS by NOESISSERVER rather than by a separate compliance function.
- Generate reusable market data: clearing prices per tier and round, and, as a byproduct of fulfillment monitoring, a record of measured provider performance over time.
- Disseminate that clearing-price data in real time to participants and downstream systems, through the pricing-dissemination component introduced in Section 1.3, rather than requiring them to poll NOESISMARKET or wait for the next round.

As in traditional exchanges, we expect the market to be compensated for this value creation through a small fee on cleared volume; we do not formalize a fee schedule in this paper.

2.3. **The bundling problem: quoting quality, not just quantity.** The central design difficulty is choosing what, precisely, is being bought and sold. A market for a homogeneous good only needs to clear on price and quantity. Machine intelligence is not homogeneous: the same nominal price per request can correspond to wildly different quality, and the same underlying compute can be used to answer the same request at very different quality depending on the model and routing policy used to serve it.

This is analogous to the base/quote asymmetry that a token-exchange protocol must resolve before a limit order is even well-defined: a limit order only makes sense once one has fixed which token plays the role of quantity and which plays the role of price. NOESIS faces a similar prior question: before a bid or an ask can be written down, the protocol must fix what a unit of "intelligence" is (Section 3.2) and what accompanies that unit as a quality specification. We adopt the position that quality should be expressed along three largely orthogonal axes:

- **Capability**: how good the answer is expected to be, expressed as a discrete tier rather than a continuous score, since tiers are what providers can credibly commit to and what buyers can reason about.
- **Latency**: an upper bound on the time to produce an answer.
- **Reliability**: the fraction of requests that meet both the capability and latency terms.

Figure 3 makes this contrast visually concrete: NOESIS's traded unit is a bundle along three largely orthogonal axes, capability, latency, and reliability, contrasted with a conventional single-scalar good quoted purely on price per token.

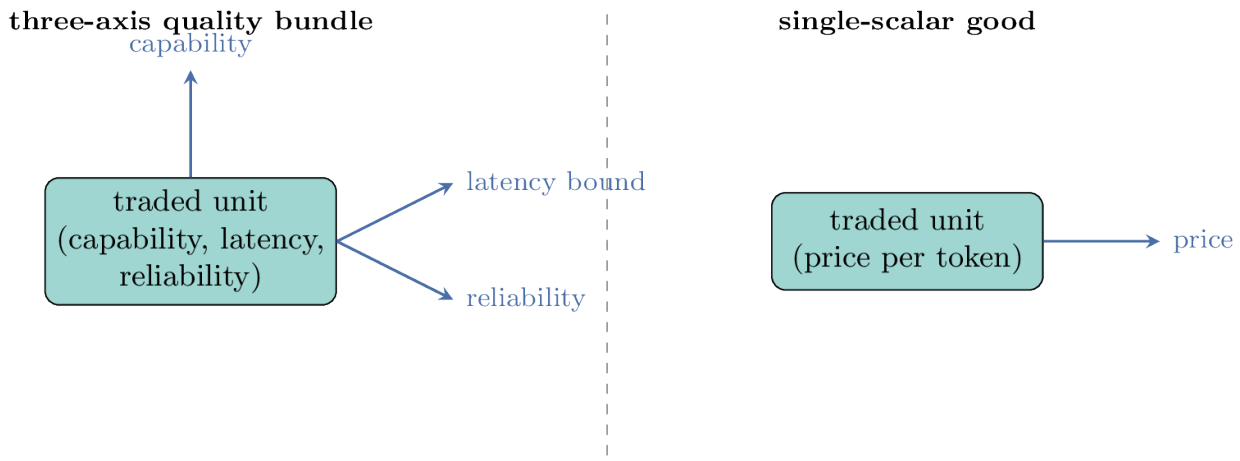


FIGURE 3. A three-axis quality bundle versus a single-scalar good.

Section 3.3 turns this into a formal contract. The consequence of this design choice is that the market cannot clear on price alone: two asks at the same price but different tiers are not substitutes, and matching must first establish compatibility along these three axes before price discovery can proceed (Section 4.2).

2.4. **Related commodity markets.** Electricity markets are the closest existing analogy. Electricity cannot be usefully stored at scale, demand and supply must be balanced continuously, and quality (voltage, frequency) is metered rather than taken on faith. Electricity markets address this with a layered design: day-ahead markets clear the bulk of expected demand a day in advance, real-time markets clear residual imbalances close to delivery, and separate capacity markets pay generators for being available rather than for energy delivered, penalizing non-delivery when it matters most [10, 11]. NOESIS's tiered call auction (Section 4) borrows the day-ahead/real-time separation as a candidate design (discussed further as an open question in Section 9), and its reputation and pricing feedback loop (Section 4.5) borrows directly from capacity-market non-performance penalties [12].

The analogy is imperfect in one important respect: electricity's quality attributes (voltage, frequency) are physical and essentially free to meter, while machine intelligence's capability attribute is not directly observable and can only be estimated from the content of a response. This is the problem that NOESISSERVER's fulfillment monitoring (Section 5.2) has to solve.

## 3. CONTRACTS AND NOTATION

This section fixes the notation used throughout the rest of the paper: the capability tier set, the task unit, the contract, and the bid/ask tuples submitted to NOESISMARKET.

### 3.1. **Capability tiers.**

**Definition 3.1** (Capability tiers)**.** Let

$$\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$$

be a finite, totally ordered set of *capability tiers*, with $c_1 \prec c_2 \prec \cdots \prec c_K$, where $\prec$ denotes "strictly less capable than". A tier is a discrete, provider-agnostic label (e.g., *cheap*, *medium*, *frontier-class*) that a buyer can require and a seller can offer, rather than a continuous benchmark score.

Tiers are deliberately coarse. A continuous capability score would be more informative but is harder for a seller to credibly commit to ex ante and harder for a buyer to reason about ex post; Section 9 returns to this trade-off.

### 3.2. **The task unit.**

**Definition 3.2** (Task)**.** A *task* is an abstract, normalized unit of inference work, denoted $u \in \mathcal{U}$, used as the unit of quantity in a contract. Throughout the formalization, quantities are expressed in tasks without committing to a specific definition of $\mathcal{U}$.

Definition 3.2 is deliberately left abstract. Candidate definitions include a raw token count, a wall-clock compute duration, or a benchmark-normalized task-equivalent that adjusts for the fact that a token from a frontier model and a token from a small model are not comparable units of work. This choice affects cross-provider comparability directly and is treated as an open research question (Section 9) rather than resolved here; the remainder of the formalization only requires that $\mathcal{U}$ support addition and a total order, both of which hold for any of the candidate definitions above.

### 3.3. **Contracts.**

**Definition 3.3** (Intelligence contract)**.** An *intelligence contract* is a tuple

$$\kappa = (\texttt{n\_tasks}, \texttt{c\_level}, \texttt{L\_max}, \texttt{R\_min}, \texttt{P})$$

where

- $\texttt{n\_tasks} \in \mathbb{N}$ is the number of tasks covered by the contract,
- $\texttt{c\_level} \in \mathcal{C}$ is the required capability tier,
- $\texttt{L\_max} \in \mathbb{R}_+$ is the maximum latency per task,
- $\texttt{R\_min} \in [0, 1]$ is the minimum reliability, i.e., the minimum fraction of tasks that must meet both `c_level` and `L_max`,
- $\texttt{P} \in \mathbb{R}_+$ is the price of the contract.

The quantities are ordered to read naturally: "deliver `n_tasks` tasks at capability `c_level` or better, within `L_max` latency, with at least `R_min` reliability, for a total price of `P`."

*Example* 3.1 (Intelligence contract)*.* The contract

$$\kappa_1 = (10{,}000,\ \texttt{frontier},\ 2\text{s},\ 0.999,\ P)$$

reads: "deliver 10,000 tasks at frontier-class capability or better, within 2 seconds of latency, with at least 99.9% reliability, for a total price of $P$."

### 3.4. **Bids and asks.**

A contract $\kappa$ is the outcome of matching a bid against one or more asks in a NoesisMarket auction round (Section 4). Bids and asks share the contract's attributes, plus the buy/sell-specific role of the price attribute as a limit rather than a fixed value.

**Definition 3.4** (Bid)**.** A *bid* is a tuple

$$\beta = (n_\beta, c_\beta, \ell_\beta, r_\beta, p_\beta)$$

submitted by a buyer, where

- $n_\beta$ is the requested number of tasks,
- $c_\beta \in \mathcal{C}$ is the minimum required capability tier,
- $\ell_\beta$ is the maximum acceptable latency,
- $r_\beta$ is the minimum acceptable reliability,
- $p_\beta$ is the maximum price per task the buyer is willing to pay.

**Definition 3.5** (Ask)**.** An *ask* is a tuple

$$\alpha = (n_\alpha, c_\alpha, \ell_\alpha, r_\alpha, p_\alpha)$$

submitted by a seller, where

- $n_\alpha$ is the offered number of tasks,
- $c_\alpha \in \mathcal{C}$ is the offered capability tier,
- $\ell_\alpha$ is the seller's typical (self-reported or previously measured) latency,
- $r_\alpha$ is the seller's typical reliability,
- $p_\alpha$ is the minimum price per task the seller is willing to accept.

Unlike a bid, an ask's capability, latency, and reliability attributes are descriptive of what the seller can deliver rather than a constraint on what the seller will accept; only $p_\alpha$ is a limit in the auction sense. Section 4.2 defines when a bid and an ask are compatible, i.e., when the ask's descriptive attributes satisfy the bid's required attributes.

## 4. NoesisMarket: the auction mechanism

NoesisMarket runs a periodic call auction that turns a pool of bids and asks into a set of intelligence contracts (Definition 3.3). This section defines auction rounds and tier bucketing, the compatibility relation between a bid and an ask, the clearing problem, the reputation and pricing feedback loop, and the mechanism-design risks the design has to withstand. The auction described throughout this section is a specific instantiation of the pluggable matching-engine component of Section 1.3; nothing below depends on the matching engine being a call auction rather than, e.g., a continuous double auction.

### 4.1. Auction rounds and tier bucketing.

Every $T$ minutes (default $T = 5$), NoesisMarket closes the open order book and runs one clearing round per capability tier $c \in \mathcal{C}$. For clarity of exposition we describe the case in which a bucket for tier $c$ contains exactly the bids with $c_\beta = c$ and the asks with $c_\alpha = c$; Remark 4.3 below discusses the natural generalization in which higher-tier asks are also eligible to fill lower-tier demand.

*Example* 4.1 (Basic auction round)*.* A buyer submits a bid for 10,000 tasks at `frontier` capability, under 2 seconds of latency, at 99.9% reliability, with a limit price $p_\beta$. Several sellers submit asks at the `frontier` tier, with various typical latencies, reliabilities, and limit prices $p_\alpha$. At the close of the round, NoesisMarket finds the compatible asks (Section 4.2), clears the `frontier` bucket at a single uniform price, and matches the buyer's bid against one or more of the compatible asks, up to 10,000 tasks.

*Remark* 4.1 (Tiered commodity pricing)*.* Because each tier clears independently, prices can diverge across tiers within the same round, in the same way that peak and off-peak electricity prices diverge within the same day. A `cheap` tier and a `frontier` tier are not substitutes from the buyer's perspective and are not expected to clear at the same price.

Figure 4 lays out this timeline: the order book accepts bids and asks continuously, and every $T$ minutes the round closes and a separate uniform-price clearing is computed independently per capability-tier bucket, so `cheap` and `frontier` can clear at different prices in the same round (Remark 4.1).

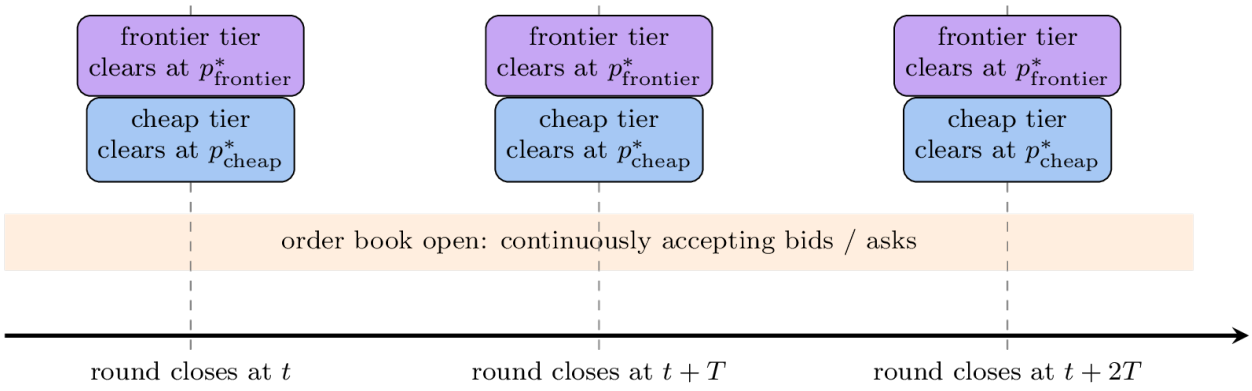


FIGURE 4. The NOESISMARKET auction timeline.

4.2. **Compatibility.** Within a tier bucket, a bid is not compatible with every ask: the bid's latency and reliability requirements still need to be met.

**Definition 4.2** (Compatibility)**.** A bid $\beta = (n_\beta, c_\beta, \ell_\beta, r_\beta, p_\beta)$ and an ask $\alpha = (n_\alpha, c_\alpha, \ell_\alpha, r_\alpha, p_\alpha)$ are *compatible*, written $\beta \sim \alpha$, if

$$c_\alpha \succeq c_\beta, \qquad \ell_\alpha \leq \ell_\beta, \qquad r_\alpha \geq r_\beta.$$

Compatibility is a feasibility filter, not a scoring function: capability, latency, and reliability are treated as hard constraints rather than as continuous inputs to the price. This is a deliberate simplification. It avoids having to define an exchange rate between, say, one unit of latency and one unit of price, at the cost of potentially leaving compatible volume unmatched when a bid's requirements are only narrowly missed; Section 9 discusses relaxing this into a scored compatibility measure.

Restricted to a tier bucket $c$, compatibility defines a bipartite graph $G_c = (B_c \cup A_c, E_c)$ between the bids $B_c$ and asks $A_c$ in the bucket, with an edge $(\beta, \alpha) \in E_c$ whenever $\beta \sim \alpha$.

Figure 5 shows a small example of such a graph: an edge marks a bid/ask pair whose capability, latency, and reliability terms are mutually satisfiable, and bid 4 has no compatible ask in this bucket and is left unmatched regardless of price.

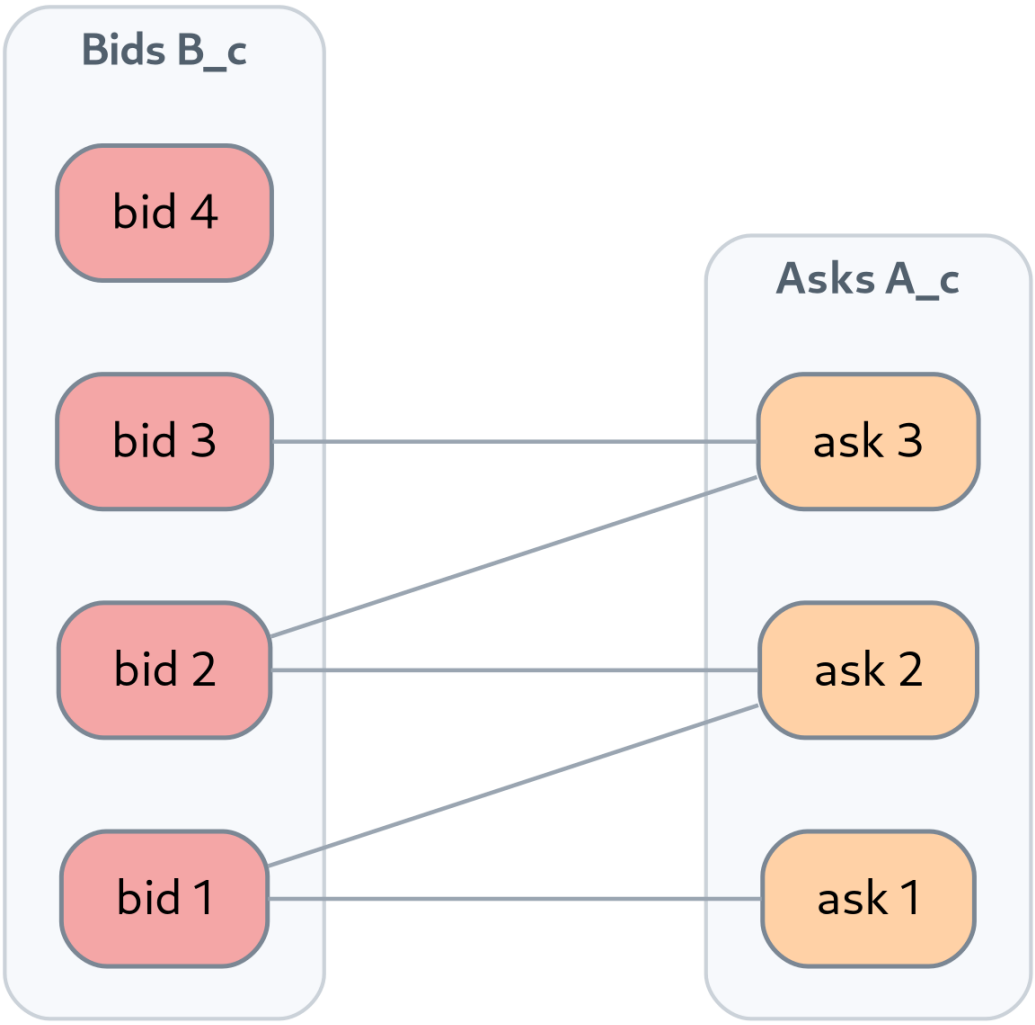


FIGURE 5. A small example compatibility graph $G_c$ within one tier bucket.

### 4.3. **The clearing problem.**

*Problem* 4.1 (NoesisMarket clearing). Fix a tier $c \in \mathcal{C}$ and round $t$, with bids $B_c$, asks $A_c$, and compatibility graph $G_c = (B_c \cup A_c, E_c)$ as above. For each edge $(\beta, \alpha) \in E_c$, let $x_{\beta\alpha} \geq 0$ denote the number of tasks matched between $\beta$ and $\alpha$. The NOESISMARKET clearing problem is to find a uniform clearing price $p^*(c,t) \in \mathbb{R}_+$ and nonnegative quantities $\{x_{\beta\alpha}\}$ that maximize the total number of matched tasks

$$\sum_{(\beta,\alpha)\in E_c} x_{\beta\alpha} \tag{1}$$

subject to

$$\sum_{\alpha\,:\,(\beta,\alpha)\in E_c} x_{\beta\alpha} \le n_\beta, \qquad \forall \beta \in B_c, \tag{2}$$

$$\sum_{\beta\,:\,(\beta,\alpha)\in E_c} x_{\beta\alpha} \le n_\alpha, \qquad \forall \alpha \in A_c, \tag{3}$$

$$x_{\beta\alpha} = 0 \text{ unless } p_\beta \ge p^*(c,t) \ge p_\alpha, \qquad \forall (\beta,\alpha) \in E_c. \tag{4}$$

Constraints (2) and (3) prevent over-filling a bid or an ask. Constraint (4) enforces the uniform-price property: at most one price clears the bucket, and only bid/ask pairs whose limit prices bracket that price may be matched, exactly as in a standard call auction [6]. Each matched bid $\beta$ gives rise to a contract $\kappa$ (Definition 3.3) with $\texttt{n\_tasks} = \sum_\alpha x_{\beta\alpha}$, $\texttt{c\_level} = c$, $\texttt{L\_max} = \ell_\beta$, $\texttt{R\_min} = r_\beta$, and $\texttt{P} = p^*(c,t) \cdot \texttt{n\_tasks}$, dispatched to NoesisServer for execution.

For fixed $p^*(c,t)$, Problem 4.1 is a transportation problem, a special case of linear programming, and is solvable in polynomial time by standard methods. Choosing $p^*(c,t)$ itself is, in general, a search over the (finite) set of distinct limit prices submitted in the bucket, since the optimal matched volume as a function of a candidate uniform price is piecewise constant.

*Remark* 4.3 (Generalization across tiers). Definition 4.2 already allows $c_\alpha \succ c_\beta$: a seller offering a higher tier than requested is compatible with a lower-tier bid. A full implementation would therefore let a tier-$c$ bucket draw on asks from tier $c$ and above, at the cost of a larger compatibility graph per round. We do not expect this to be exploited by rational sellers in practice, since serving a lower tier's demand at that tier's clearing price is only attractive when the seller's marginal cost is below it, but the possibility is worth flagging since it is exactly the kind of cross-tier fungibility that a naive per-tier bucketing could miss.

### 4.4. **Existence of a clearing price.**

**Proposition 4.4** (Existence of a clearing price)**.** *Consider a tier bucket $c$ in which the compatibility graph $G_c$ is complete, i.e., every bid in $B_c$ is compatible with every ask in $A_c$ (for instance, because every ask in the bucket meets the most demanding latency and reliability requirement among the bids in the bucket). Let $D(p) = \sum_{\beta\in B_c : p_\beta \ge p} n_\beta$ denote cumulative demand at price $p$ and $S(p) = \sum_{\alpha\in A_c : p_\alpha \le p} n_\alpha$ denote cumulative supply at price $p$. Since $D$ is non-increasing and $S$ is non-decreasing in $p$, there exists a price $p^*(c,t)$ at which the volume-maximizing matching of Problem 4.1 equals $\min(D(p^*), S(p^*))$, and no price supports strictly more matched volume.*

*Proof sketch.* Under a complete compatibility graph, Problem 4.1 reduces to a standard uniform-price double auction: for any candidate price $p$, the maximum feasible matched volume is $\min(D(p), S(p))$, achieved by matching the highest-limit-price compatible bids against the lowest-limit-price compatible asks. As $p$ increases, $D(p)$ is non-increasing and $S(p)$ is non-decreasing, so $\min(D(p), S(p))$ is maximized at any $p^*$ where the two curves cross (or, in the discrete case, at the price step immediately before $D$ falls below $S$). This is the classical Walrasian argument for existence of a market-clearing price [6]. □

When the compatibility graph is not complete, Problem 4.1 remains a well-posed linear program for any fixed candidate price, but a single uniform price is no longer guaranteed to support a volume-maximizing matching with zero unmatched compatible pairs; in that case, $p^*(c,t)$ is a design choice among the prices that support the (still well-defined) volume-maximizing matching, for instance the limit price of the marginal cleared ask.

4.5. **Reputation and pricing feedback loop.** Every contract $\kappa$ dispatched to NoesisServer for execution is later reported back with a measured reliability $\hat{r}(\kappa, W)$ over its fulfillment window

$W$ (Section 5.2). NoesisMarket maintains a reputation score per seller and uses it to gate participation in future rounds. As with the matching engine, the specific update rule given below is one instantiation of the pluggable reputation-and-feedback component of Section 1.3.

**Definition 4.5** (Seller reputation)**.** For a seller whose ask $\alpha$ was matched into contract $\kappa$ at round $t$, the reputation score $\rho_\alpha(t) \in [0, 1]$ is updated as

$$\rho_\alpha(t+1) = (1-\lambda)\,\rho_\alpha(t) + \lambda \cdot \mathbb{1}[\hat{r}(\kappa, W) \geq r_\kappa]\,, \tag{5}$$

where $\lambda \in (0, 1]$ is a decay rate controlling how quickly recent performance dominates the score.

A seller with $\rho_\alpha(t) < \rho_{\min}$, for a market-wide threshold $\rho_{\min}$, is excluded from submitting asks in the tier where the shortfall occurred, or is required to submit asks at a lower tier reflecting the downgraded assessment of its actual capability. This mirrors the non-performance penalties used in electricity capacity markets to keep generators accountable for the capacity they committed to sell [12].

*Example* 4.2 (Default). A seller wins a contract at the `frontier` tier with `R_min` $= 0.999$, but NoesisServer measures $\hat{r}(\kappa, W) = 0.97$ over the fulfillment window. The violation is reported to NoesisMarket, which lowers $\rho_\alpha(t)$ via Equation (5); if $\rho_\alpha$ falls below $\rho_{\min}$, the seller is excluded from the `frontier` bucket in subsequent rounds until its reputation recovers.

Figure 6 traces this loop from a cleared contract back to future eligibility: a contract $\kappa$ dispatched to NoesisServer yields a measured reliability $\hat{r}(\kappa, W)$, which drives the reputation update of Equation (5), and the resulting score gates the seller's eligibility to submit asks in future NoesisMarket rounds, closing the loop.

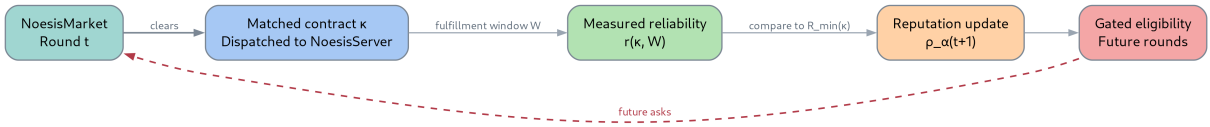


Figure 6. The NoesisMarket reputation and pricing feedback loop.

4.6. **Mechanism-design risks.** The design above leaves at least three classical mechanism-design problems open, which we flag here and revisit in Section 9.

- **Capability misrepresentation**: an ask may overstate $c_\alpha$ to be eligible for a higher-clearing-price bucket than the seller can actually serve. Noesis does not attempt to prevent this ex ante; it is deterred ex post through the reputation and pricing feedback loop of Section 4.5, once fulfillment monitoring detects the shortfall.
- **Bid shading**: uniform-price double auctions are not, in general, dominant-strategy incentive compatible, so a rational buyer or seller may misreport its true valuation [8]. Dominant-strategy mechanisms exist for restricted settings, such as single-unit double auctions [6], but do not obviously extend to the multi-unit, multi-attribute setting of Problem 4.1 without efficiency loss.
- **Collusion**: a thin tier bucket, with few sellers, is vulnerable to the kind of coordinated bidding studied in the auction literature on bidding rings [9], which is more of a concern at the `frontier` tier, where the number of credible providers is small by construction.

We do not claim to resolve these problems in this paper; the auction design in Problem 4.1 is a starting point whose robustness to strategic behavior is left for empirical and theoretical follow-up work.

## 5. NoesisServer: fulfillment and monitoring

NoesisServer is the layer that turns a matched contract (Definition 3.3) into actual inference requests against real providers, and turns the resulting traffic into three things: a measurement of whether the contract was honored, a signal for difficulty-aware routing, and a corpus for downstream distillation.

5.1. **Gateway architecture.** NoesisServer is, at its core, a lightweight LLM API gateway modeled on multi-provider routers such as OpenRouter: it exposes a single API, proxies each request to one or more underlying providers, and streams the response back to the caller. Two properties distinguish it from a plain pass-through router. First, every request is tagged with the contract $\kappa$ it is served under, if any, so that fulfillment can be attributed (Section 5.2). Second, every prompt/response pair is logged with its provider, model, measured latency, measured cost, and, when available, a user feedback signal, regardless of whether the request was placed under a contract. The storage schema is intentionally minimal:

$$(\text{prompt}, \text{response}, \text{provider}, \text{model}, \text{latency}, \text{cost}, \kappa \text{ or } \bot)$$

This logging is passive by default: it does not change routing behavior and exists purely to build a reusable dataset for evaluation and distillation (Section 5.5). Privacy and data-handling safeguards (PII scrubbing, opt-in/opt-out) are necessary before logging real traffic and are discussed as an open question in Section 9.

Figure 7 traces this path end to end: the caller's request is proxied to a provider and the response streamed back, and on the way back, the request is logged as the tuple (prompt, response, provider, model, latency, cost, contract-or-$\bot$) described above.

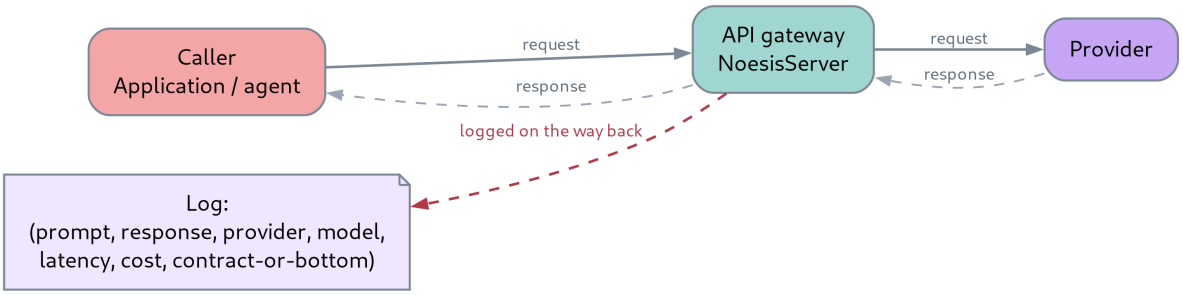


Figure 7. Path of a single request through NoesisServer.

5.2. **Contract fulfillment monitoring.** For a request $x$ served under contract $\kappa$, let $\hat{c}(x) \in \mathcal{C}$ denote the capability tier actually delivered and $\hat{\ell}(x)$ the actually measured latency, both estimated from the response (e.g., via a verifier model, a benchmark probe, or agreement with a stronger reference model). This estimator is the pluggable capability-measurement component of Section 1.3: the fulfillment test below only requires $\hat{c}(x)$, not a commitment to one specific way of computing it. Request $x$ is a *success* under $\kappa$ if

$$s(x) = \mathbb{1}\Big[\hat{c}(x) \succeq \texttt{c\_level}(\kappa) \ \wedge \ \hat{\ell}(x) \leq \texttt{L\_max}(\kappa)\Big]. \tag{6}$$

**Definition 5.1** (Measured reliability)**.** For a fulfillment window $W$, the set of requests served under contract $\kappa$, the measured reliability is

$$\hat{r}(\kappa, W) = \frac{1}{|W|} \sum_{x \in W} s(x). \tag{7}$$

A naive violation rule, flagging $\kappa$ whenever $\hat{r}(\kappa, W) < \texttt{R\_min}(\kappa)$, conflates genuine under-delivery with sampling noise: $\hat{r}(\kappa, W)$ is a sample average of an unobserved true success rate, and a small $|W|$ makes false accusations likely. We instead treat fulfillment monitoring as a one-sided hypothesis test: the null hypothesis $H_0$ is that the seller is compliant, i.e., its true success rate is at least $\texttt{R\_min}(\kappa)$, and $\kappa$ is flagged as violated only if $H_0$ is rejected at a chosen confidence level $1 - \delta$. A simple normal-approximation lower confidence bound gives

$$\hat{r}_{\text{lower}}(\kappa, W) = \hat{r}(\kappa, W) - z_{1-\delta} \sqrt{\frac{\hat{r}(\kappa, W)\,(1 - \hat{r}(\kappa, W))}{|W|}}, \tag{8}$$

where $z_{1-\delta}$ is the corresponding standard normal quantile, and $\kappa$ is flagged as violated when $\hat{r}_{\text{lower}}(\kappa, W) < \texttt{R\_min}(\kappa)$. For small $|W|$, an exact interval (e.g., Clopper–Pearson) is preferable to the normal approximation used here for exposition. Flagged violations are reported to NoesisMarket and feed the reputation update of Equation (5).

*Example* 5.1 (Fulfillment reporting)*.* A contract promising 99.9% reliability at `frontier` capability is served by three providers over a window of 5,000 requests. NoesisServer computes $\hat{r}(\kappa, W) = 0.994$ and, applying Equation (8), finds $\hat{r}_{\text{lower}}(\kappa, W) < 0.999$: the shortfall is unlikely to be sampling noise, so the contract is flagged as violated and reported back to NoesisMarket.

Figure 8 places the point estimate, its lower confidence bound, and the threshold on a single number line for both outcomes of the test: when the confidence interval $[\hat{r}_{\text{lower}}, \hat{r}(\kappa, W)]$ stays entirely above $\texttt{R\_min}(\kappa)$, the shortfall, if any, is attributed to sampling noise, and when the interval dips below $\texttt{R\_min}(\kappa)$, $\kappa$ is flagged as violated and reported to NoesisMarket.

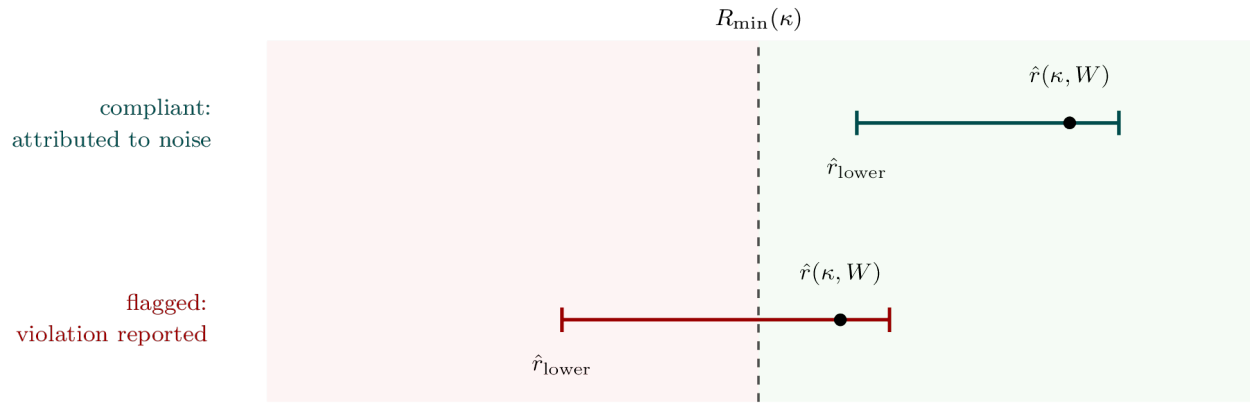


Figure 8. The reliability hypothesis test on a single number line.

5.3. **Difficulty-aware routing.** Not every request needs the strongest available model. NoesisServer trains a difficulty estimator

$$\hat{d} : \mathcal{P} \to \mathcal{C} \tag{9}$$

mapping a prompt $x \in \mathcal{P}$ to the cheapest capability tier expected to answer it correctly, and routes according to

$$\pi(x) = \mathsf{min}\{c \in \mathcal{C} : c \succeq \hat{d}(x)\}. \tag{10}$$

Routing is only worthwhile if the estimator is cheaper than the savings it produces. Let $\mathrm{cost}(c)$ denote the per-request cost of serving tier $c$, non-decreasing in $c$, let $c_K$ denote the strongest tier, and let $c_{\hat{d}}$ denote the per-request cost of evaluating $\hat{d}$ itself. Routing yields a net saving over an always-route-to-$c_K$ baseline exactly when

$$c_{\hat{d}} + \mathbb{E}_x[\,\mathrm{cost}(\pi(x))\,] < \mathrm{cost}(c_K). \tag{11}$$

Inequality (11) makes explicit the trap flagged in the underlying design notes: an estimator that is itself expensive, or inaccurate enough to route hard requests to weak tiers and force costly retries, can cost more than always calling the strong model. Measuring the left-hand side against the right-hand side on real traffic is a prerequisite for deploying routing in production, and is deferred to future implementation work (Section 9).

Figure 9 lays out this chain next to the condition under which it pays off: a prompt $x$ is mapped by the difficulty estimator to the cheapest capability tier $\pi(x)$ expected to answer it correctly, and routing only pays off when the estimator's own cost plus the expected routed cost undercuts always calling the strongest tier (Inequality (11)).

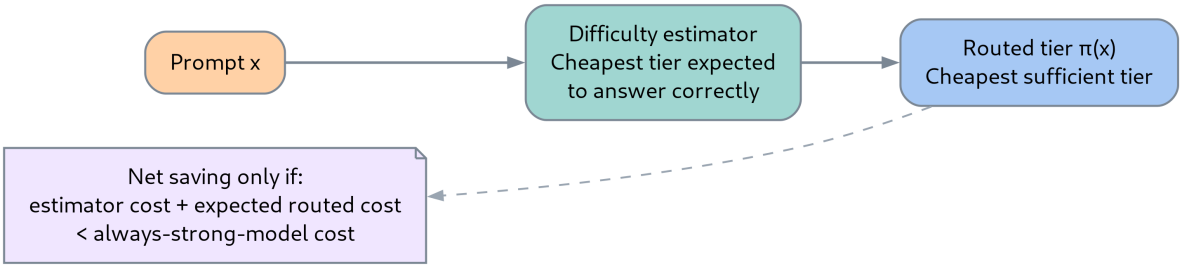


FIGURE 9. Difficulty-aware routing to the cheapest sufficient capability tier.

5.4. **Answer fusion.** Fusion is the complementary lever to routing. Instead of picking one tier per request, NOESISSERVER may query a set of providers $S(x) \subseteq \mathcal{M}$ and combine their answers through an aggregator

$$\hat{y}(x) = g\big(\{f_m(x)\}_{m \in S(x)}\big), \tag{12}$$

where $g$ may be a majority vote, a verifier model that scores and selects among candidates, or a learned combiner trained on logged outcomes: the aggregator is the pluggable answer-fusion component of Section 1.3.

Routing and fusion trade cost for accuracy in opposite directions relative to a single-strong-model baseline: routing accepts a higher risk of a lower-quality answer on some fraction of requests in exchange for a lower expected cost, while fusion accepts a higher expected cost, from querying multiple providers per request, in exchange for a lower risk of a low-quality answer. Which lever is preferable for a given contract depends on the contract's price versus its `R_min` term: a low-price, low-`R_min` contract favors routing, while a high-`R_min` contract may only be economically served through fusion.

Figure 10 contrasts fusion's fan-out/fan-in shape with routing's single-provider path: fusion sends the same prompt to every provider in $S(x)$ and combines their answers through the aggregator $g$ (Equation (12)), trading extra inference cost for a lower risk of a low-quality answer.

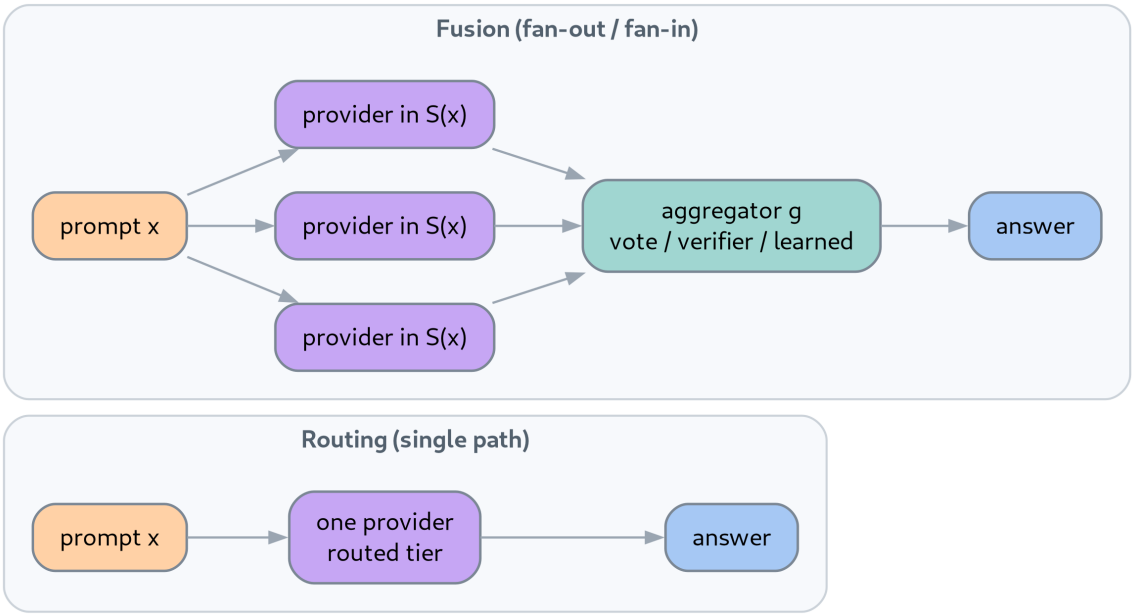


FIGURE 10. Answer fusion's fan-out/fan-in versus routing's single-provider path.

5.5. **Training signal and distillation.** The difficulty estimator of Equation (9) needs a training signal, which the logged traffic supplies for free.

**Definition 5.2** (Agreement label)**.** For a prompt $x$ answered by both a cheap model $f_{\text{cheap}}$ and a strong model $f_{\text{strong}}$, the agreement label is

$$y(x) = 1 - \mathbb{1}\big[f_{\text{cheap}}(x) \equiv f_{\text{strong}}(x)\big], \tag{13}$$

where $\equiv$ denotes exact match or a similarity threshold, used as a proxy label for "this prompt was hard" ($y(x) = 1$) versus "easy" ($y(x) = 0$).

Definition 5.2 is a proxy, not a ground-truth correctness label: the two models can agree on a wrong answer, or disagree while the cheap model happens to be correct. A genuine quality label requires independent verification, for instance from a pairwise preference arena, which is a companion research direction discussed in Section 6. With that caveat, the logged corpus

$$\mathcal{D} = \{(x_i, f_{\text{strong}}(x_i))\}_{i=1}^{N}$$

also supports knowledge distillation of a smaller model $f_\theta$ trained to approximate $f_{\text{strong}}$'s outputs on the observed task distribution [5], offering a second, independent way for NOESISSERVER to reduce cost on traffic it has already observed.

Figure 11 shows the two consumers that share this one logged dataset: logged traffic feeds agreement labels (Definition 5.2), which train the difficulty estimator of Section 5.3, while the same

traffic also forms the corpus $\mathcal{D}$ used to distill a smaller model $f_\theta$ (Section 5.5), as two parallel, independent consumers of one dataset.

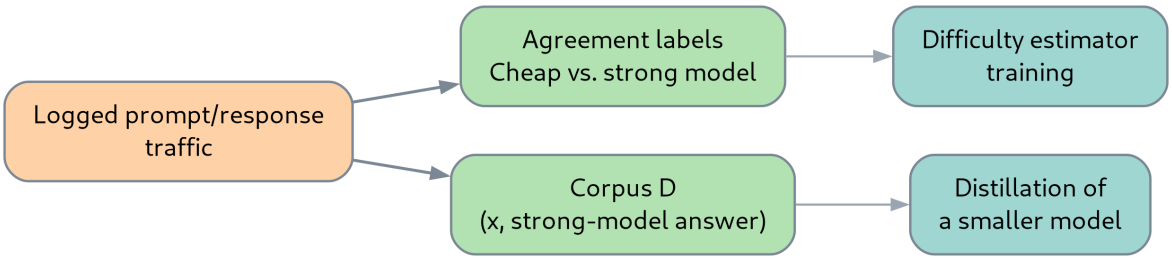


FIGURE 11. The training pipeline built from one logged dataset.

## 6. Related work

6.1. **LLM gateways and routing.** Multi-provider gateways such as OpenRouter unify access to many LLM providers behind a single API, which is the architectural starting point for NoesisServer (Section 5.1); unlike Noesis, they typically set prices unilaterally rather than clearing an auction, and do not meter delivered quality against a contract. RouteLLM [1] and FrugalGPT [2] address the difficulty-aware routing problem of Section 5.3 directly, learning to route or cascade requests across models of different cost to reduce spend while controlling quality loss; Noesis adopts a similar routing formalization but ties the resulting cost savings to contract fulfillment and to a market that prices the tiers being routed between.

6.2. **Model fusion and distillation.** Self-consistency [3] and mixture-of-agents [4] are examples of the answer-fusion strategy of Section 5.4: combining multiple model outputs, by voting or by a further aggregation model, to trade additional inference cost for higher answer quality. Knowledge distillation [5] underlies the training-signal reuse of Section 5.5, in which NoesisServer's logged traffic plays the role of the training corpus.

6.3. **Auction and market microstructure.** The uniform-price call auction of Problem 4.1 is a multi-attribute generalization of the classical double auction [6]. The choice of a periodic batch auction, rather than a continuous limit order book, follows the argument that frequent batch auctions remove the latency-arbitrage incentives that plague continuous markets [7], an argument we expect to carry over to a market where sellers could otherwise race to snipe favorably priced demand. The strategic risks flagged in Section 4.6, bid shading and collusion, draw on the classical auction-theory literature on optimal and incentive-compatible auction design [8] and on collusive bidding [9].

6.4. **Commodity and capacity market design.** Electricity market design [10, 11] is the closest analogy to Noesis, as discussed in Section 2.4: the day-ahead/real-time separation motivates the tiered, periodic clearing of Section 4.1, and non-performance penalties in capacity markets [12] motivate the reputation and pricing feedback loop of Section 4.5.

6.5. **Decentralized exchange protocols.** Noesis's contract, bid, and ask notation (Section 3) and its treatment of the clearing problem as a linear program (Problem 4.1) follow the approach taken by Tulip, a protocol for decentralized token exchange built around limit orders and a Walrasian-style auction [13]. The two protocols differ in an important respect: Tulip's traded

good, a token, is settled entirely on-chain and requires no external verification, whereas Noesis's traded good, a unit of intelligence, is produced by off-chain computation whose quality can only be estimated, not settled atomically, which is why NoesisServer's fulfillment monitoring (Section 5.2) has no counterpart in Tulip. Whether NoesisMarket's contract matching and settlement, as opposed to the inference computation itself, should move on-chain, in the manner of Tulip or of automated market makers such as Uniswap [15, 16], is sketched as a concrete, blind-bid, stake-backed design in Section 8, drawing on the companion ZK-Tulip design notes [14]; the resulting open questions are consolidated in Section 9.

Figure 12 places the work surveyed above on these two axes: routing/fusion work and auction theory each occupy one axis; electricity and capacity markets are the closest existing analogy, occupying both, but for a physically metered commodity rather than machine intelligence (Section 6.4); Noesis is the only point combining both axes for machine intelligence.

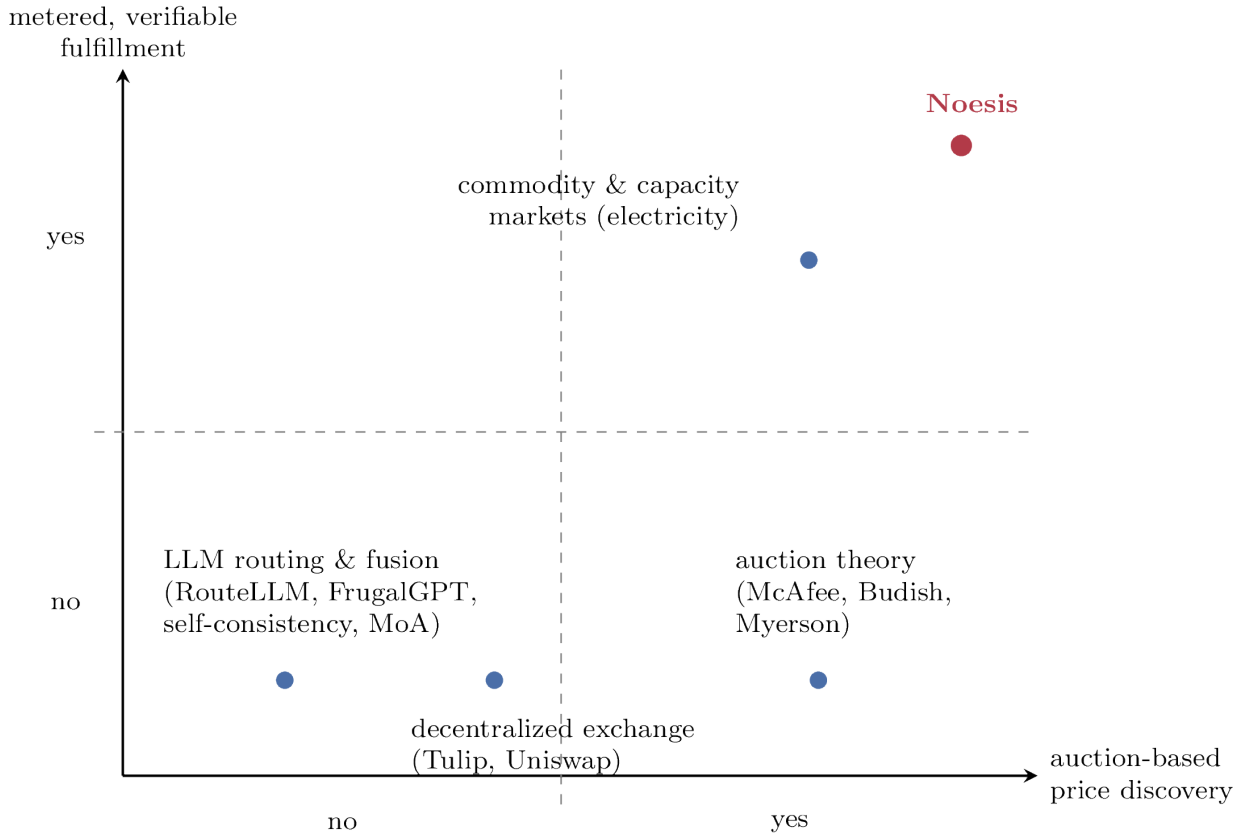


Figure 12. Related work positioned by auction-based price discovery and metered fulfillment.

6.6. **Companion research directions.** Three ongoing, unpublished research notes are directly relevant and are referenced throughout this paper without being cited as external publications. A blockchain-based prediction market shares NoesisMarket's matching-and-settlement problem, without the capability-verification problem that motivates NoesisServer. A pairwise preference arena for LLMs would supply the independent, ground-truth quality judgments that Definition 5.2's agreement label only approximates, and could be used to score routing and fusion decisions directly. A dataset-construction effort for training and distilling LLMs would be a direct consumer of the corpus logged by NoesisServer (Section 5.5).

## 7. State of the art: industry landscape

Section 6 positions Noesis against academic work on routing, fusion, distillation, and auction theory. This section instead surveys currently operating startups and projects that address, piece by piece, the same problem Noesis targets: buying and selling machine intelligence, trading raw compute, or verifying that a delivered service met its promise. The goal is to locate the specific combination of properties, auction-based price discovery for a capability/latency/reliability bundle, metered fulfillment, and reputation feedback into pricing, that no surveyed system currently occupies. The industry moves quickly, so the picture below reflects the state of the market at the time of writing and should be read as illustrative of the categories rather than exhaustive within any one of them.

7.1. **Posted-price multi-provider gateways.** OpenRouter, introduced in Section 1.2, is one of several gateways that unify access to hundreds of models behind a single API and bill at or near the underlying provider's price. Portkey, LiteLLM, Requesty, TrueFoundry's AI Gateway, Cloudflare AI Gateway, and the Vercel AI Gateway occupy the same niche, adding a fixed platform fee, a flat markup, or no markup at all on top of pass-through provider pricing, together with observability, caching, and automatic failover across providers. A single-vendor gateway such as AWS Bedrock offers the same unified-API convenience without the multi-seller pooling. OpenRouter additionally ships a mixture-of-agents feature, Fusion, that fans a prompt out to several models and synthesizes one answer, which is the closest commercial analog to the answer-fusion strategy of Section 5.4 that this survey identified. Table 3 lists the systems in this cluster.

| | *URL* | *Distinguishing feature* |
|---|---|---|
| OpenRouter | `https://openrouter.ai` | Multi-vendor pooling; ships Fusion mixture-of-agents synthesis |
| Portkey | `https://portkey.ai` | Pass-through pricing + platform fee/markup† |
| LiteLLM | `https://litellm.ai` | Pass-through pricing + platform fee/markup† |
| Requesty | `https://requesty.ai` | Pass-through pricing + platform fee/markup† |
| TrueFoundry AI Gateway | `https://truefoundry.com` | Pass-through pricing + platform fee/markup† |
| Cloudflare AI Gateway | `https://developers.cloudflare.com/ai-gateway` | Pass-through pricing + platform fee/markup† |
| Vercel AI Gateway | `https://vercel.com/ai-gateway` | Pass-through pricing + platform fee/markup† |
| AWS Bedrock | `https://aws.amazon.com/bedrock` | Single-vendor; unified API without multi-seller pooling |

Table 3. Posted-price multi-provider gateways surveyed in Section 7.1. †Also bundles observability, caching, and automatic failover across providers.

None of these gateways clears an auction: every price is posted or passed through unilaterally, so there is no price discovery in the sense of Section 1.5. None meters whether a request's capability, latency, or reliability matched a stated contract beyond a standard infrastructure uptime SLA, and none turns a delivered shortfall into a reputation signal that changes a provider's standing in future transactions.

7.2. **Learned model routers.** A second cluster of startups routes each individual request to the model expected to give the best cost/quality trade-off, which is the commercial counterpart to the difficulty-aware routing of Section 5.3. Martian routes on an interpretability-derived model of what

each request needs. Not Diamond and Unify learn or benchmark a routing policy across models on standard evaluation suites. Arch, an open-source, Envoy-based prompt gateway, routes from a natural-language policy description rather than hardcoded rules. Table 4 lists the systems in this cluster.

| | *URL* | *Distinguishing feature* |
|---|---|---|
| Martian | `https://withmartian.com` | Routes on an interpretability-derived model of request needs |
| Not Diamond | `https://notdiamond.ai` | Learns/benchmarks a routing policy on standard evaluation suites |
| Unify | `https://unify.ai` | Learns/benchmarks a routing policy on standard evaluation suites |
| Arch | `https://github.com/katanemo/archgw` | Open-source, Envoy-based; routes from a natural-language policy description |

TABLE 4. Learned model routers surveyed in Section 7.2.

These routers make the same difficulty-aware bet as NOESISSERVER, that most requests do not need the most capable model, but the routing decision is a private policy internal to each vendor rather than the outcome of a market that prices the tiers being routed between, and none of them exposes a reputation signal that a request's actual capability, latency, or reliability fell short of what was routed for.

7.3. **Reverse-auction and spot compute marketplaces.** A third cluster trades raw compute, rather than model access, through genuine bidding mechanisms. Akash Network runs a reverse auction in which a tenant posts a workload specification and a budget and providers bid to fill it. Vast.ai runs a continuous bid/ask market for GPU-hours in which an underbid spot instance can be reclaimed by a higher bidder. RunPod, io.net, Golem Network, CoreWeave, and Prime Intellect offer spot, reserved, or brokered tiers over pooled GPU capacity, with pricing that ranges from genuinely biddable to posted with a spot discount. Table 5 lists the systems in this cluster.

| | *URL* | *Bid-based* | *Distinguishing feature* |
|---|---|---|---|
| Akash Network | `https://akash.network` | + | Reverse auction: tenant posts workload + budget, providers bid |
| Vast.ai | `https://vast.ai` | + | Continuous bid/ask GPU-hour market; underbid spot instances reclaimable |
| RunPod | `https://runpod.io` | o | Spot/reserved/brokered GPU tiers[†] |
| io.net | `https://io.net` | o | Spot/reserved/brokered GPU tiers[†] |
| Golem Network | `https://golem.network` | o | Spot/reserved/brokered GPU tiers[†] |
| CoreWeave | `https://coreweave.com` | o | Spot/reserved/brokered GPU tiers[†] |
| Prime Intellect | `https://primeintellect.ai` | o | Spot/reserved/brokered GPU tiers[†] |

TABLE 5. Reverse-auction and spot compute marketplaces surveyed in Section 7.3.
[†]Pricing ranges from genuinely biddable to posted with a spot discount.

Akash and Vast.ai in particular demonstrate that an open bid/ask market for compute-time is viable at production scale, which is direct evidence for the auction half of NOESISMARKET's design.

The traded unit, however, is a GPU-hour or a workload's raw resource footprint, not the (capability, latency, reliability) bundle of Section 1.1: none of these markets prices, or even observes, how good the resulting model answer is, so the bundled-quality and difficulty-routing rows of Table 2 remain unaddressed.

7.4. **Token-incentivized inference networks.** A fourth cluster builds reputation and verification directly into a decentralized compute network, which is the closest analog to the fulfillment-monitoring and reputation loop that connects NoesisServer and NoesisMarket (Sections 5.2 and 4.5). Bittensor organizes miners and validators into subnets; in its inference-oriented subnets, validators send prompts to many miners, score the returned completions for quality and latency, and an on-chain consensus mechanism aggregates the stake-weighted scores into recurring reward emissions, with a serverless-inference subnet layered on top for pay-per-query access. Gensyn verifies distributed training work through a probabilistic proof-of-learning protocol rather than scoring inference output. Ritual runs a network of inference oracles that attach a cryptographic proof of correct execution to each computation. Nosana and the Fetch.ai / ASI Alliance pool idle compute and agent-to-agent transactions for AI workloads more broadly. Table 6 lists the systems in this cluster.

| | *URL* | *Distinguishing feature* |
|---|---|---|
| Bittensor | `https://bittensor.com` | Validator scoring of inference quality/latency; stake-weighted reward emissions |
| Gensyn | `https://gensyn.ai` | Probabilistic proof-of-learning verifies distributed training |
| Ritual | `https://ritual.net` | Inference oracles attach a cryptographic proof of correct execution |
| Nosana | `https://nosana.com` | Pools idle compute and agent-to-agent transactions for AI workloads |
| Fetch.ai / ASI Alliance | `https://fetch.ai` | Pools idle compute and agent-to-agent transactions for AI workloads |

Table 6. Token-incentivized inference networks surveyed in Section 7.4.

Bittensor's validator scoring is the only mechanism surveyed here that continuously measures output quality and lets that measurement change a provider's economic standing, which is precisely the loop Section 4.5 formalizes for NoesisMarket. It differs from Noesis in two respects: the reward is an emission schedule internal to the network's consensus rather than a price cleared against an explicit buyer bid, and the score is a continuous internal ranking rather than a pass/fail check against a specific bought (volume, tier, latency, reliability) contract. Gensyn's and Ritual's proofs verify that a computation was executed correctly, not that its output met a stated capability or latency bound, so they address a different, complementary notion of trust.

7.5. **Compute exchanges and derivatives.** A fifth, more recent cluster builds explicit auction or index infrastructure on top of the raw-compute markets of Section 7.3. SF Compute operates an order-book-style market for GPU-time blocks. Compute Exchange runs spot auctions over standardized compute contracts explicitly modeled on a commodity exchange. Andromeda brokers capacity across many providers on standardized, benchmarked contracts to avoid multi-year lock-in. Ornn's Compute Price Index publishes a transaction-based benchmark price for GPU rental, feeding into early cash-settled compute futures. Table 7 lists the systems in this cluster.

| | *URL* | *Distinguishing feature* |
|---|---|---|
| SF Compute | `https://sfcompute.com` | Order-book-style market for GPU-time blocks |
| Compute Exchange | `https://compute.exchange` | Spot auctions over standardized compute contracts |
| Andromeda | `https://andromeda.ai` | Brokers capacity across providers on standardized, benchmarked contracts |
| Ornn | `https://ornn.ai` | Compute Price Index: transaction-based benchmark price feeding cash-settled futures |

TABLE 7. Compute exchanges and derivatives surveyed in Section 7.5.

This cluster is the strongest evidence that the pairing of an open auction with a standardized contract, the structural bet underlying NOESISMARKET, is a fundable design for AI infrastructure: it did not exist a few years ago and now spans spot auctions, brokered standardized contracts, and price indices. Every instance found, however, prices raw compute capacity rather than a verified LLM answer, so none of them closes the fulfillment-to-reputation loop that Section 5 adds on top of the market.

7.6. **Serverless inference providers as prospective sellers.** A final cluster, serverless inference providers such as Together AI, Fireworks AI, Groq, Baseten, and Fal.ai (Table 8), sell inference at posted per-token or per-second rates at a given capability tier. These are not competitors to NOESIS so much as its prospective sellers: each already meters its own delivered latency and offers a standard infrastructure SLA, which is a useful starting point for the request-level fulfillment statistics of Section 5.2, but none of them exposes that metering as a contract a buyer can shop across sellers for, or as a reputation signal that follows the seller into a future transaction.

| | *URL* |
|---|---|
| Together AI | `https://together.ai` |
| Fireworks AI | `https://fireworks.ai` |
| Groq | `https://groq.com` |
| Baseten | `https://baseten.co` |
| Fal.ai | `https://fal.ai` |

TABLE 8. Serverless inference providers surveyed in Section 7.6, each selling inference at posted per-token or per-second rates at a given capability tier.

7.7. **Comparison and market gap.** Table 9 extends the comparison of Table 2 to the six clusters surveyed above, each represented by its most characteristic mechanism.

Figure 13 plots the six clusters on these two axes: gateways, learned routers, and serverless providers set prices unilaterally; compute marketplaces and exchanges bid on raw compute but neither price nor observe answer quality; token-incentivized networks verify quality without a market clearing price against a buyer's bid; no surveyed cluster combines both properties for machine intelligence, and NOESIS occupies that combination.

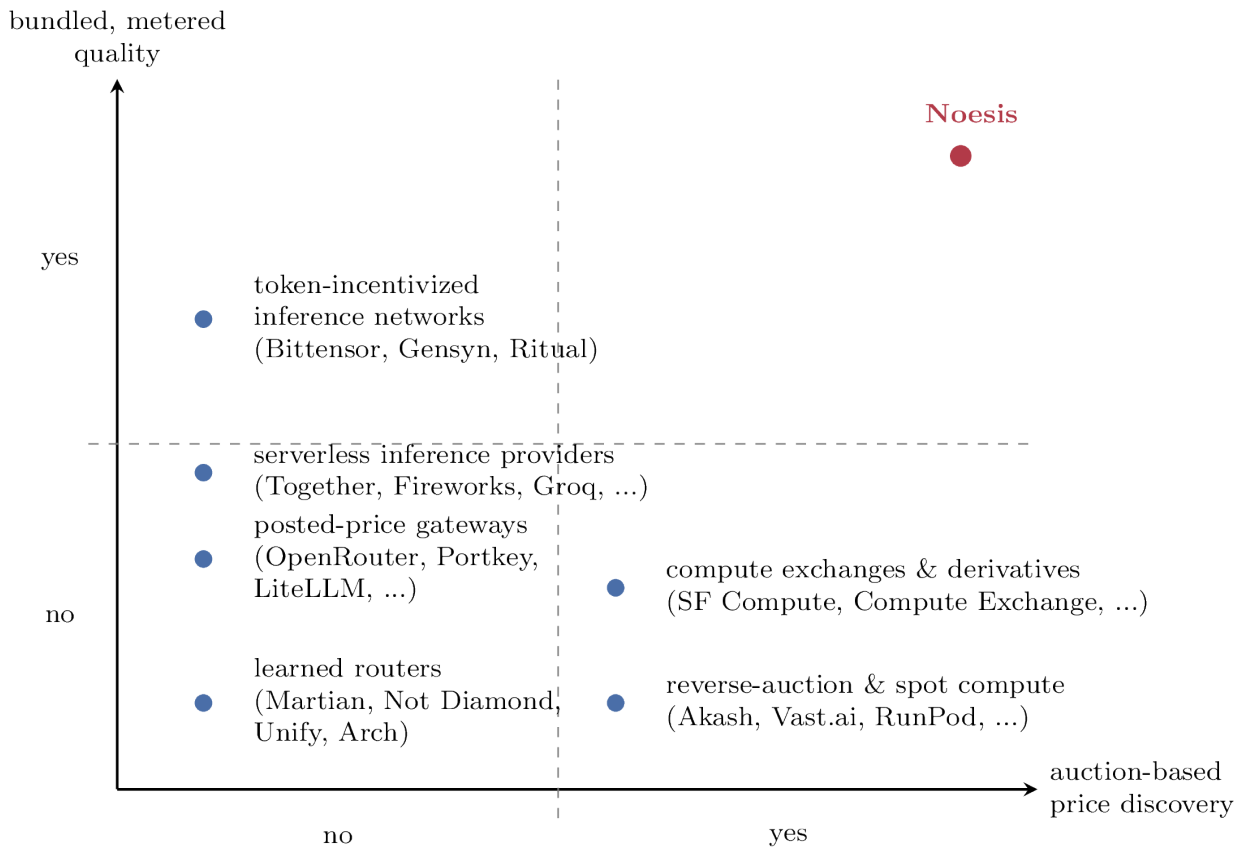


FIGURE 13. The six industry clusters positioned by price discovery and bundled quality.

| | *Auction price* | *Bundled quality* | *Metered fulfillment* | *Reputation → pricing* | *Liquidity pooling* |
|---|---|---|---|---|---|
| Posted-price gateways (§7.1) | - | o | - | - | + |
| Learned model routers (§7.2) | - | o | - | - | o |
| Reverse-auction compute markets (§7.3) | + | - | - | o | + |
| Token-incentivized inference networks (§7.4) | - | - | o | + | o |
| Compute exchanges & derivatives (§7.5) | + | - | o | o | + |
| Serverless inference sellers (§7.6) | - | o | - | - | - |
| NOESIS | + | + | + | + | + |

TABLE 9. Comparison of NOESIS against the six industry clusters surveyed in Section 7. "Bundled quality" denotes a purchasable (capability, latency, reliability) contract rather than a raw compute quantity or a single posted price; "metered fulfillment" denotes verification of delivered quality against that contract, rather than a generic infrastructure SLA.

No cluster combines auction-based price discovery with a bundled, verifiable quality contract and a reputation loop that feeds back into pricing. What is striking is that each individual ingredient is already validated in production by at least one system: Akash and Vast.ai show that an open auction for compute is viable; Compute Exchange and Andromeda show that standardizing the traded contract attracts liquidity; Bittensor shows that continuously scoring delivered quality and tying it

to economic standing changes provider incentives; OpenRouter's Fusion feature shows that answer fusion is a sellable product feature. NOESIS's contribution is not any one of these mechanisms in isolation, each already has a real-world precedent, but their combination into a single loop, applied to a verifiable (capability, latency, reliability) bundle rather than to raw compute, which to our knowledge no surveyed system currently offers.

## 8. Toward a decentralized, anonymized NoesisMarket

Sections 4 and 5 describe NOESIS as operated by a single, trusted market and gateway. That design is deliberately the simplest starting point: it lets the auction (Section 4) and the fulfillment layer (Section 5) be specified without also solving distributed-systems and cryptographic-protocol problems. This section sketches an alternative in which NOESISMARKET's auction runs on a public blockchain rather than inside a single operator's infrastructure, with bids and asks kept anonymous until a round closes. The design adapts ZK-Tulip [14], an unpublished working note on a stake-based, blind-bid marketplace for zero-knowledge proof generation: NOESISMARKET's clearing problem (Problem 4.1) and ZK-Tulip's proof-matching problem share the same shape, matching a requester who needs a unit of off-chain computation performed under a deadline against a supplier who is paid for performing it, so the mechanisms below carry over with only minor changes. Section 7.4 surveys production systems, Bittensor, Gensyn, and Ritual among them, that already run parts of this machinery, validator scoring, proof-of-learning, proof-of-execution, at network scale; the question this section asks is how far analogous machinery can go when applied to NOESISMARKET's own auction and contract, rather than to a network-internal reward schedule.

We treat what follows as a design sketch rather than a specification. Section 8.7 collects the open questions it leaves unresolved, most of which are open questions in ZK-Tulip's own design notes as well.

### 8.1. **Motivation: censorship resistance and bidder privacy.**

The centralized design of Section 4 concentrates two kinds of trust in a single operator. First, the operator sees every bid and ask, including a buyer's required capability tier and volume, which can reveal competitively sensitive information, for instance that a buyer is about to run a large batch job, before the round even clears. Second, the operator alone decides which asks are eligible to compete, which contracts clear, and which sellers are excluded on reputation grounds (Section 4.5), with no mechanism for a participant to independently verify that this was done correctly.

A blockchain-based NOESISMARKET addresses both concerns by moving the parts of the mechanism that most need to be trust-minimized, contract terms, matching, settlement, and nonperformance penalties, onto a public ledger, while keeping bid and ask contents hidden until the round closes (Section 8.2). Neither property is free: on-chain settlement trades the negligible latency and cost of a centralized clearinghouse for the latency, gas cost, and finality assumptions of the underlying chain, a trade-off Section 8.6 makes explicit.

Figure 14 contrasts the two trust boundaries before the mechanisms below are introduced: in the centralized design, a single operator sees every bid and ask in full and alone decides eligibility and matching, while in the decentralized design, only on-chain addresses are visible until the reveal window closes (Section 8.2), and matching and eligibility are verifiable on-chain rather than taken on the operator's word.

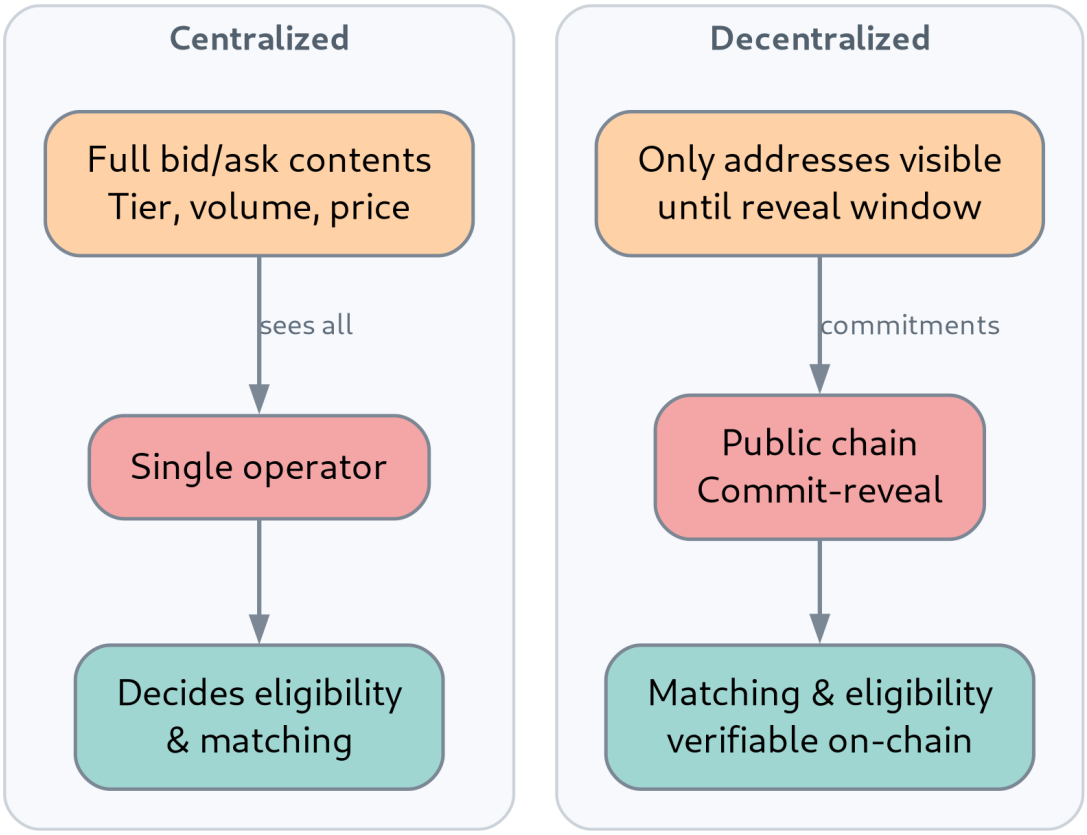


FIGURE 14. The two trust boundaries: centralized versus decentralized NOESISMARKET.

8.2. **On-chain contracts, blind bidding, and anonymity.** The contract, bid, and ask notation of Section 3 carries over unchanged: a decentralized round still clears (`n_tasks`, `c_level`, `L_max`, `R_min`, `P`) tuples (Definition 3.3). What changes is how a bid or an ask is submitted and revealed.

**Definition 8.1** (Blind bid)**.** A *blind bid* for a bid $\beta = (n_\beta, c_\beta, \ell_\beta, r_\beta, p_\beta)$ (Definition 3.4) is a commitment

$$h_\beta = H(\beta \,\|\, \nu_\beta)$$

posted on-chain during a round's *commit window*, where $H$ is a collision-resistant hash function and $\nu_\beta$ is a secret salt known only to the bidder. During the round's subsequent *reveal window*, the bidder discloses $(\beta, \nu_\beta)$; only bids and asks revealed within that window, and matching a previously posted commitment, are eligible for the clearing problem (Problem 4.1) run at the round's close. An ask $\alpha$ is committed and revealed symmetrically.

Definition 8.1 follows the commit-reveal pattern already used by ZK-Tulip's blind-bid mechanism, itself motivated by the same concern that motivates frequent batch auctions over continuous limit order books (Section 6.3): a party that can see pending bids before a round closes can front-run

them, either by submitting a marginally better ask to snipe a favorable price, or, at the block-production layer, by reordering or censoring specific bids. Committing before revealing reduces the incentive to do so, at the cost of one additional round-trip and the mempool-monitoring risk flagged in Section 8.7: a party can still observe bids as they are revealed and submit a competing bid before the reveal window closes, unless that window is itself no longer than a single block.

Anonymity is a byproduct of the same design: a buyer and a seller are identified on-chain only by an address, and, until the reveal window, not even the contents of their bid are visible. This does not, by itself, resolve NoesisServer's own privacy question, what safeguards are needed before logging real prompt/response pairs (Section 5.1): fulfillment still requires a buyer's requests to reach a real seller off-chain, so anonymity here covers the auction, not the inference traffic that a cleared contract subsequently generates.

Figure 15 traces one round through these three phases: during the commit window, bidders post the commitment $h_\beta = H(\beta \,\|\, \nu_\beta)$ (Definition 8.1); during the reveal window, they disclose $(\beta, \nu_\beta)$; and only revealed bids and asks matching a posted commitment are kept for the clearing problem (Problem 4.1) run at the round's close.

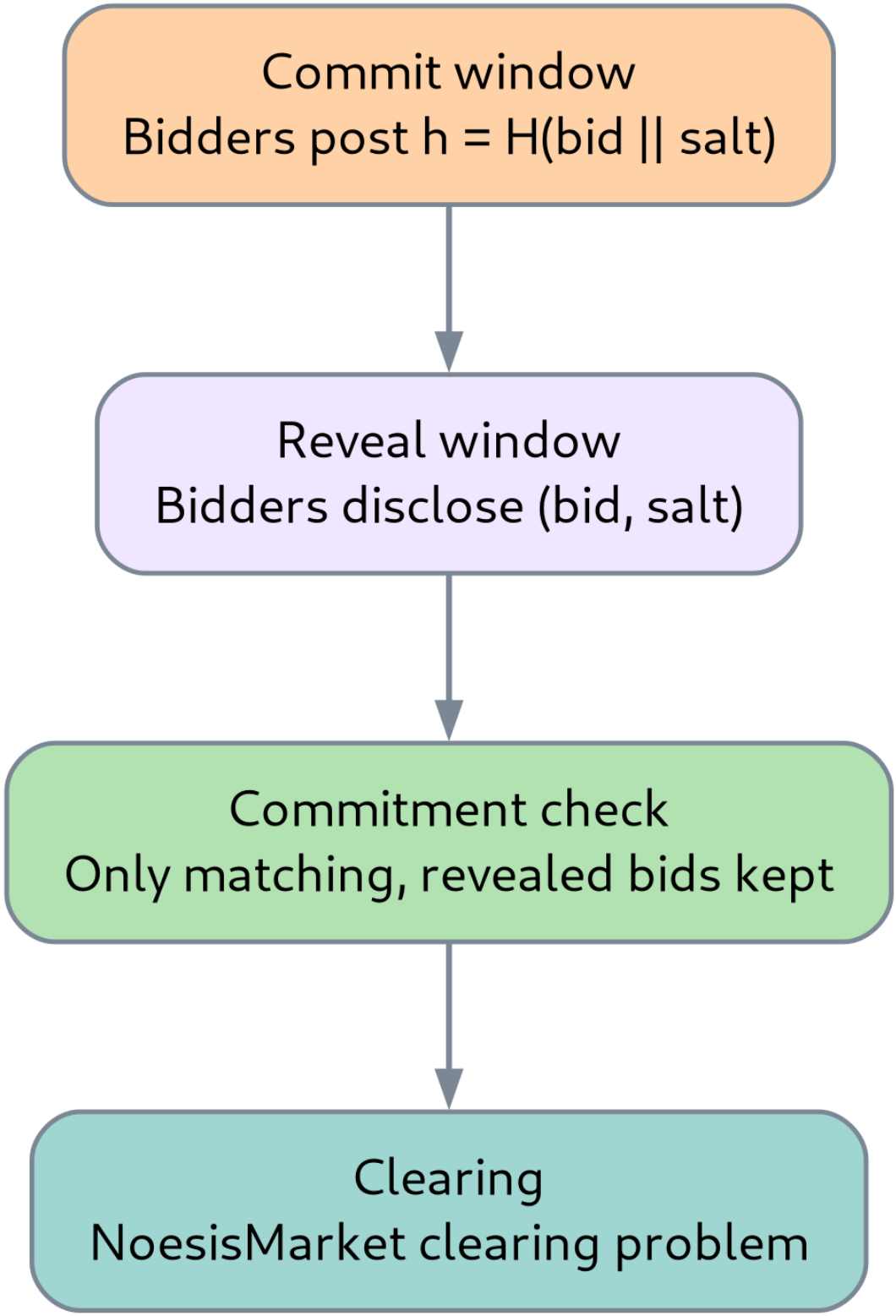




FIGURE 15. One auction round under commit-reveal.

8.3. **Stake-backed fulfillment: slashing as decentralized reputation.** Section 4.5's reputation score $\rho_\alpha(t)$ is a soft, ex post penalty enforced by a trusted operator: a seller who under-delivers is downgraded, but only the operator can act on that downgrade. A decentralized NoesisMarket replaces, or complements, this with a hard, cryptoeconomic penalty.

**Definition 8.2** (Staked ask)**.** A *staked ask* extends an ask $\alpha$ (Definition 3.5) with a stake $\sigma_\alpha \in \mathbb{R}_+$, denominated in a fixed token $\tau$, posted to an escrow contract at the time $\alpha$ is revealed. If $\alpha$ is matched into a contract $\kappa$, the stake is locked until the fulfillment window $W$ closes (Section 5.2). It is released in full if $\kappa$ is not flagged as violated, i.e., $\hat{r}_{\text{lower}}(\kappa, W) \geq \texttt{R\_min}(\kappa)$ (Equation (8)), and slashed, in full or in part, otherwise.

Definition 8.2 still needs the report of $\hat{r}_{\text{lower}}(\kappa, W)$ itself, computed off-chain by NoesisServer from delivered responses; slashing does not remove the need for fulfillment monitoring, it changes what happens once a violation is detected, from a reputation downgrade to an immediate, automatic, on-chain forfeiture. This is a stronger deterrent against the capability misrepresentation risk of Section 4.6 precisely because it does not require the seller's continued participation in future rounds to take effect. How $\hat{r}_{\text{lower}}(\kappa, W)$ is reported on-chain in a form a smart contract can act on, rather than treating NoesisServer itself as a trusted oracle, is left open in Section 8.7.

Figure 16 shows this lifecycle as a state diagram: the stake is posted when $\alpha$ is revealed, locked once matched into a contract $\kappa$, and released in full or slashed once $\hat{r}_{\text{lower}}(\kappa, W)$ is reported against $\texttt{R\_min}(\kappa)$ at the close of the fulfillment window.

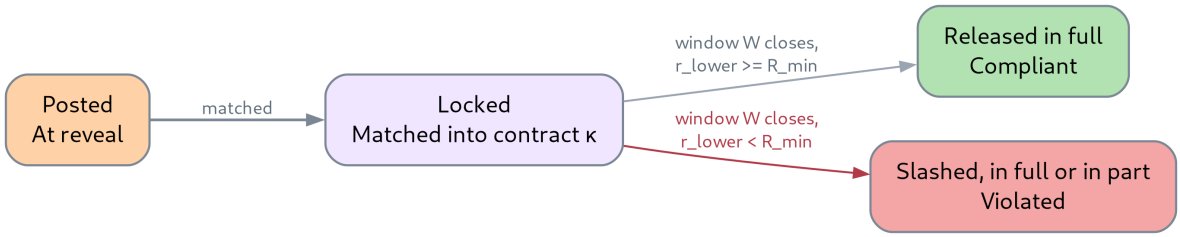


FIGURE 16. Lifecycle of a staked ask.

Sizing $\sigma_\alpha$ raises the same asymmetry ZK-Tulip's design notes flag for its own per-bid stake: a buyer setting the required stake unilaterally can grief sellers by demanding collateral far in excess of the contract's value, while a buyer who is not itself bonded has nothing at risk if a matched contract turns out not to be genuine. Section 8.7 returns to this.

8.4. **Where the clearing problem runs.** Problem 4.1 is a linear program over the revealed bids and asks in a tier bucket. Solving it naively on-chain, recomputing the clearing price and every matched quantity $x_{\beta\alpha}$ inside a smart contract, scales the gas cost of a round with $|B_c| \cdot |A_c|$, which is unattractive for a bucket with more than a handful of participants. Three placements are possible.

- **Fully on-chain**: the clearing problem is solved inside the smart contract itself. Trust-minimized by construction, but the gas cost above makes it impractical beyond small buckets.
- **Fully off-chain**: a designated operator solves Problem 4.1 off-chain and posts only the result, $p^*(c,t)$ and the matched contracts. Cheap, but reintroduces the single trusted operator that Section 8.1 set out to remove: nothing on-chain prevents the operator from posting an incorrect or self-favoring solution.

– **Verifiably off-chain**: an off-chain solver computes $p^*(c,t)$ and $\{x_{\beta\alpha}\}$ as before, but additionally produces a succinct proof $\phi$ that the solution satisfies the feasibility constraints of Problem 4.1 (Equations (2)–(4)) and leaves no compatible pair unmatched at a price both sides would accept. A smart contract verifies $\phi$ in time independent of $|B_c| + |A_c|$ and only then finalizes the round.

The third option is the one ZK-Tulip's design notes gesture at when they ask whether the matchmaking engine can run partially off-chain with a zero-knowledge property showing the solution is correct, and it is the option that preserves both the trust-minimization goal of Section 8.1 and the practicality of the fully-off-chain design. It is also the least developed of the three: producing a succinct, efficiently verifiable proof of optimality for a linear assignment problem, rather than merely of feasibility, is itself an open implementation question, listed in Section 8.7.

Figure 17 spans the three placements along this spectrum, from trust-minimized and expensive to cheap and trusted: the verifiably-off-chain placement is the least developed of the three but is the only one that preserves trust-minimization without the gas cost of solving Problem 4.1 on-chain.

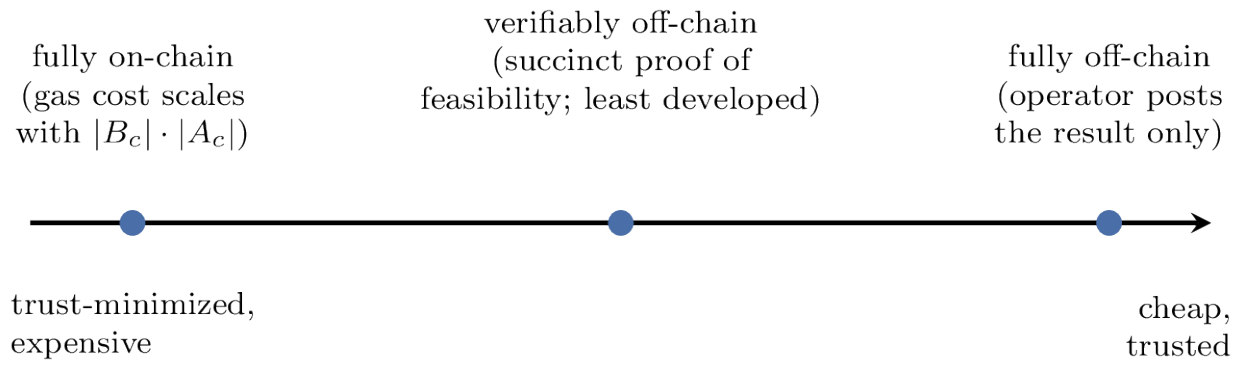


FIGURE 17. The three placements for the clearing problem, on a trust/cost spectrum.

8.5. **Redundancy as a decentralization safeguard.** A single seller's downtime is a single point of failure for the contract that seller was matched into. ZK-Tulip's design notes address the analogous risk, a single prover failing to deliver a proof in time, with a redundancy parameter: a requester can choose to have $k_{\text{red}} \geq 1$ provers race for the same job, each staking collateral, with only the first, or, in an auction variant, the top few, to deliver being paid.

The same parameter adapts directly to a decentralized NOESISMARKET: a contract can be matched against $k_{\text{red}}$ staked asks instead of one, with traffic split or raced across them during the fulfillment window, and only the sellers who meet the contract's terms retaining their stake. This is a different lever from answer fusion (Section 5.4): fusion queries several providers per request to improve the quality of a single answer, while redundancy here queries several providers to hedge against any one of them being unavailable or slashed, and is a settlement-layer property rather than an answer-quality one. Redundancy also gives the market a tool against the concentration risk noted in Section 4.6's discussion of thin, collusion-prone tier buckets: instead of always matching a buyer's full volume against the single lowest-priced compatible ask, a decentralized NOESISMARKET can split it across the $k_{\text{red}}$ lowest-priced compatible asks, so that the same top seller does not deterministically win every round in a thin bucket.

Figure 18 contrasts a redundant match with the single-ask baseline: instead of matching a contract against a single staked ask, it is matched against $k_{\text{red}}$ staked asks that race to fulfill it,

and only the sellers who meet the contract's terms retain their stake, hedging against any one seller being unavailable or slashed.

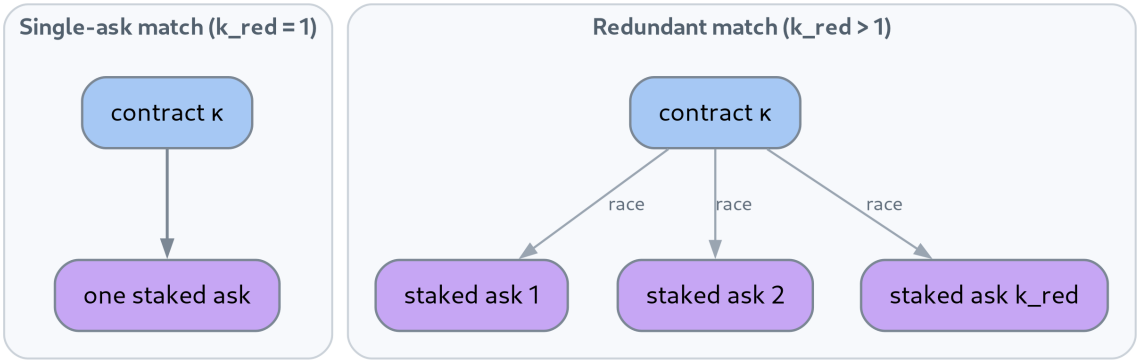


FIGURE 18. Redundancy as a decentralization safeguard.

8.6. **Comparison with the centralized design.** Table 10 summarizes the trade-off between the centralized NOESISMARKET of Section 4 and the decentralized variant sketched above.

| | *Centralized* NOESISMARKET | *Decentralized* NOESISMARKET |
|---|---|---|
| Clearing requires a trusted operator | - | + |
| Bidder identity and volume kept private | - | + |
| Non-performance penalty is automatic | o | + |
| Settlement latency and cost | + | o |
| Capability quality independently verified | - | - |

TABLE 10. Trade-offs between the centralized NOESISMARKET of Section 4 and the decentralized, blind-bid variant sketched in this section.

The last row is deliberately unchanged across both designs: staking (Section 8.3) makes non-fulfillment costly, but it does not, by itself, make the delivered capability $\hat{c}(x)$ (Section 5.2) any more verifiable than it is in the centralized design. This is the same gap Section 6.5 identifies relative to Tulip: a token transfer settles atomically on-chain, but an LLM response's quality is estimated, not settled, whichever design clears the contract that produced it.

8.7. **Open questions specific to the decentralized design.**

1. **Bid and price discovery for a new market**: with no trading history, neither a buyer nor a seller has a reference price to anchor a bid or an ask to; whether the first rounds of a new tier bucket need a different mechanism, e.g., a wider price band or a subsidized bootstrap period, than the steady-state design above is unresolved.
2. **Reveal-window griefing**: a blind bid (Definition 8.1) is visible to on-chain observers once revealed; a party monitoring the mempool during the reveal window can still submit a competing bid or ask before the window closes, unless the window is bounded to a single block, which in turn bounds how many participants can feasibly reveal in time.

3. **Commitment without capability**: nothing in Definition 8.1 stops a seller from committing an ask it cannot actually fulfill within `L_max`, since the stake $\sigma_\alpha$ is only forfeited after the fact; whether a smaller, non-refundable commitment fee at bid time is needed to discourage speculative asks is open.
4. **Stake sizing and asymmetry**: whether the stake $\sigma_\alpha$ required of a seller (Section 8.3) should be set unilaterally by the buyer's bid, which risks the buyer demanding excessive collateral, or by a market-wide rule tied to the contract's price, and whether the buyer should also be required to post a bond so that an under-collateralized buyer cannot costlessly request contracts it does not intend to honor payment for.
5. **Sybil resistance**: whether stake-and-slash (Section 8.3) is by itself a sufficient deterrent against a seller re-entering under a new address after being slashed, or whether a Sybil-resistant identity layer is still needed on top, in tension with the anonymity goal of Section 8.1.
6. **Redundancy defaults**: whether $k_{\text{red}}$ (Section 8.5) should default to 1, as in ZK-Tulip's design notes, and be raised only at the buyer's request and expense, or whether the market itself should raise it automatically in thin, collusion-prone buckets.
7. **Verifiable optimality**: whether a succinct proof of clearing-problem optimality, not merely feasibility (Section 8.4), is practical to generate and verify at the scale of a real tier bucket, or whether the design must settle for verified feasibility and accept a centralized operator's word on optimality.

## 9. Open questions

The design in Sections 4 and 5 is a starting point rather than a settled protocol. This section consolidates the open questions raised throughout the paper, grouped by which component they primarily affect.

### 9.1. Market design questions.

1. **Task unit**: which definition of the task unit (Definition 3.2), raw tokens, wall-clock compute, or a benchmark-normalized task-equivalent, gives the best cross-provider comparability, and how sensitive is the auction outcome to this choice?
2. **Auction frequency and mechanism**: is a uniform-price call auction on a fixed $T$-minute cadence (Section 4.1) the right mechanism at this scale, or would a continuous double auction serve latency-sensitive buyers better, mirroring the tension between frequent batch auctions and continuous limit order books [7]? A hybrid design, a fast spot market layered on top of the periodic batch auction, mirroring the day-ahead/real-time separation in electricity markets (Section 2.4), is a natural candidate we have not evaluated.
3. **Compatibility versus scoring**: Definition 4.2 treats latency and reliability as hard eligibility constraints. Would a scored compatibility measure, admitting partial matches at a discount, recover otherwise-unmatched volume without requiring an explicit exchange rate between quality and price?
4. **Pricing denomination**: should `P` be denominated in a real-world currency or in a synthetic task-credit, and how much does that choice affect adoption and regulatory exposure?
5. **Resistance to gaming without gatekeeping**: how far can Section 4.6's ex post reputation penalty substitute for ex ante seller vetting, without the market drifting toward the kind of slow onboarding or reputation gate that would defeat the goal of an open market?
6. **On-chain settlement and anonymity**: Section 8 sketches a decentralized, blind-bid variant of NoesisMarket; whether that design's gas costs, redundancy defaults, and stake asymmetries (Section 8.7) are compatible with the auction frequency of Section 4.1 remains open.

9.2. **Server design questions.**

1. **Where the value actually lies**: is a simple pass-through logger, without any routing intelligence, already useful for building a dataset for downstream distillation, or does routing quality itself materially affect the diversity and usefulness of the collected data?
2. **Net savings from routing**: how cheaply can the difficulty estimator of Equation (9) be trained and served while still satisfying the saving condition of Equation (11), and what fraction of real traffic can be served by a small model with no measurable quality loss?
3. **Routing versus fusion under a fixed budget**: for a fixed per-request budget, is it preferable to route to one well-chosen model (Section 5.3) or to query several cheaper models and combine their answers (Section 5.4)?
4. **Privacy and data handling**: what safeguards, PII scrubbing, opt-in versus opt-out consent, retention limits, are required before NoesisServer logs real prompt/response pairs at scale (Section 5.1)?
5. **Distillation quality**: can a model distilled from the logged corpus (Section 5.5) match the quality of the routed "best tier per request" policy at a fraction of the inference cost, or does distillation only recover the easy-request tail?
6. **Attribution reliability**: the hypothesis test of Equation (8) distinguishes sampling noise from a true reliability shortfall, but it still relies on $\hat{c}(x)$ and $\hat{\ell}(x)$ being accurate per-request estimates of delivered capability and latency; how reliable can those per-request estimates be made, and how does verifier error propagate into false or missed violations reported to NoesisMarket?

9.3. **Cross-cutting questions.** The reputation and pricing feedback loop of Section 4.5 is only as trustworthy as the fulfillment monitoring that feeds it (Section 5.2). Resolving the server-side attribution-reliability question above is therefore a prerequisite, not an independent improvement, for the market-side reputation mechanism to have the intended deterrent effect on capability misrepresentation and under-delivery.

**Interface contracts for pluggable components**: Section 1.3 treats the matching engine, capability measurement, reputation and feedback, answer fusion, and pricing dissemination as independently replaceable components, but this paper only sketches each interface in prose. Making a component genuinely swappable, e.g., trying a continuous double auction in place of the call auction of Section 4 without touching NoesisServer, requires pinning down the exact inputs, outputs, and invariants each interface guarantees, which is left as future specification work rather than resolved here.

## 10. Conclusion

We have described Noesis, a protocol that treats machine intelligence as a fungible, but not homogeneous, commodity, and that organizes its exchange around two cooperating components: NoesisMarket, a periodic call auction that clears bids and asks bundling capability, latency, and reliability into a single contract, and NoesisServer, the gateway that executes those contracts, meters whether they were fulfilled, and reuses the resulting traffic for difficulty-aware routing, answer fusion, and distillation. Each of these two components is, in turn, a thin coordination layer around independently pluggable sub-components, matching engine, capability measurement, reputation and feedback, answer fusion, and pricing dissemination (Section 1.3), rather than a fixed monolith. We formalized the contract and auction notation, stated the market-clearing problem as a linear program, established existence of a clearing price under a completeness assumption on the compatibility graph, and proposed a hypothesis-testing approach to distinguishing genuine contract violations from sampling noise.

The design suggests that the two problems, price discovery for a bundled quality good and verified fulfillment of that good, are naturally complementary: a market without fulfillment monitoring has no way to enforce what was sold, and a fulfillment layer without a market has no price signal to feed back into. Section 8 sketches one further direction: a blockchain-based, blind-bid variant of NoesisMarket that trades the operational simplicity of a centralized clearinghouse for censorship resistance and bidder anonymity, at the cost of the gas, latency, and stake-sizing questions consolidated in Section 9.

We have not, however, evaluated this design against real traffic, real providers, or strategic sellers, and Section 4.6 makes clear that the mechanism's robustness to bid shading, capability misrepresentation, and collusion is an open question rather than a settled property. The open questions of Section 9, including the interface contracts each pluggable component of Section 1.3 would need to be truly swappable, are intended as a concrete, incremental path toward closing that gap, starting from components, an auction simulator and a logging proxy, that are each independently useful and testable before the full closed loop is attempted.

## References


[1] Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665*.

[2] Chen, L., Zaharia, M., & Zou, J. (2023). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*.

[3] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2022). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *arXiv preprint arXiv:2203.11171*.

[4] Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., & Zou, J. (2024). Mixture-of-Agents Enhances Large Language Model Capabilities. *arXiv preprint arXiv:2406.04692*.

[5] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

[6] McAfee, R. P. (1992). A Dominant Strategy Double Auction. *Journal of Economic Theory*, 56(2), 434–450.

[7] Budish, E., Cramton, P., & Shim, J. (2015). The High-Frequency Trading Arms Race: Frequency Arbitrage and the Design of Financial Markets. *Quarterly Journal of Economics*, 130(4), 1547–1621.

[8] Myerson, R. B. (1981). Optimal Auction Design. *Mathematics of Operations Research*, 6(1), 58–73.

[9] McAfee, R. P., & McMillan, J. (1992). Bidding Rings. *American Economic Review*, 82(3), 579–599.

[10] Stoft, S. (2002). *Power System Economics: Designing Markets for Electricity*. IEEE Press / Wiley-Interscience.

[11] Cramton, P. (2017). Electricity Market Design. *Oxford Review of Economic Policy*, 33(4), 589–612.

[12] Cramton, P., & Stoft, S. (2005). A Capacity Market that Makes Sense. *The Electricity Journal*, 18(7), 43–54.

[13] Anonymous (2023). Tulip: A Decentralized Protocol for Token Exchange. Unpublished working paper.

[14] Anonymous (2023). ZK-Tulip: A Stake-Based, Blind-Bid Marketplace for Zero-Knowledge Proof Generation. Unpublished working note.

[15] Adams, H., Zinsmeister, N., & Robinson, D. (2020). Uniswap v2 Core. Whitepaper.

[16] Angeris, G., & Chitra, T. (2020). Improved Price Oracles: Constant Function Market Makers. *arXiv preprint arXiv:2003.10001*.